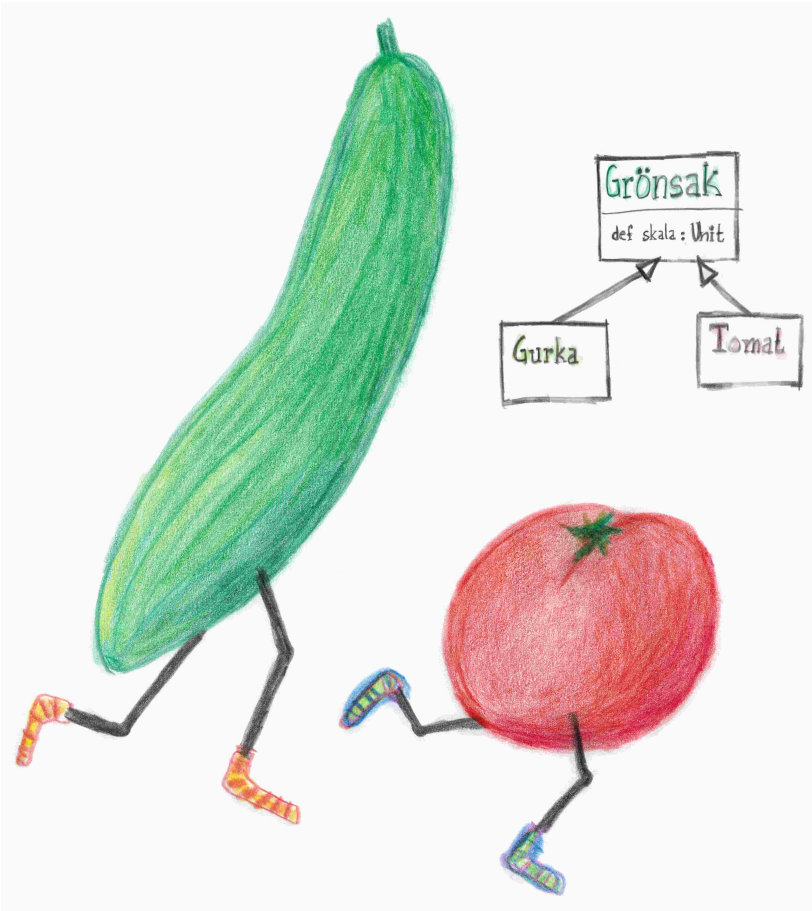
# Introduction to programming with Scala

Solutions to exercises




H-ICT-SE-2-25 2026
Object Oriented Software Development
The Hague University of Applied Science

Compilation date: April 16, 2026
https://lunduniversity.github.io/pgk

# Contents


1. Solution Excercise01 . . . . . 2
2. Solution Excercise02 . . . . . 17
3. Solution Excercise03 . . . . . 26
4. Solution Excercise04 . . . . . 34
5. Solution Excercise05 . . . . . 45
6. Solution Excercise06 . . . . . 57
7. Solution Excercise07 . . . . . 66
8. Solution Excercise08 . . . . . 80
9. Solution Excercise09 . . . . . 88
10. Solution Excercise10 . . . . . 93
11. Solution Excercise11 . . . . . 102
12. Solution Excercise12 . . . . . 111
13. Solution Excercise13 . . . . . 121

# 1. Solution `Excercise01`

## 1.1 Basic Tasks; preparation for the lab

**Solution Task 1.** *Match concepts with description.*

| | | | | |
|---|---|---|---|---|
| literal | 1 | ⇝ | D | represents a specific data value |
| string | 2 | ⇝ | G | a sequence of characters |
| statement | 3 | ⇝ | F | a line of code that does something; can be separated with a semicolon |
| expression | 4 | ⇝ | H | combines values and functions into a new value |
| function | 5 | ⇝ | K | computes a return value when called |
| procedure | 6 | ⇝ | J | performs a side effect when called; return value is empty |
| runtime error | 7 | ⇝ | N | may occur while the program runs |
| compile error | 8 | ⇝ | M | may occur before execution starts |
| abstract | 9 | ⇝ | A | introduce new concepts that simplify coding |
| compile | 10 | ⇝ | C | translate code into executable form |
| type | 11 | ⇝ | I | describes what the data can be used for |
| for-loop | 12 | ⇝ | O | useful when the number of repetitions is known beforehand |
| while-loop | 13 | ⇝ | P | useful when the number of repetitions is unknown beforehand |
| assignment | 14 | ⇝ | L | to change the value of a variable |
| floating-point | 15 | ⇝ | E | decimal number with limited precision |
| boolean | 16 | ⇝ | B | either true or false |

**Solution Task 2.** *Output in Scala REPL.*

a) For example:

```
scala> println("hello lasser")
```

b) If the right parenthesis is missing you can continue typing on the next line. This is indicated by a vertical bar at the start of each new line:

```
scala> println("hello lasser"
     | + "!"
     | )
hello lasser!
```

**Solution Task 3.** *String concatenation.*

a)

```
scala> "cucumber" + "tomato"
res1: String = cucumbertomato
```

value: `"cucumbertomato"`, type: `String`

b)

```
scala> res1 * 42
```

```
res2: String = cucumbertomatocucumbertomatocucumbertomatocucumbertomatocucumber
```

**Solution Task 4.** *When is the error detected?*

a) Type: `String`, value: `"hejhejhej"`

b) Runtime error:

```
scala> "hej" * Int.MaxValue
java.lang.OutOfMemoryError: Java heap space
```

c) Compile-time error: (indicated by text `<console> ... error:`)

```
scala> "hej" * true
<console>:12: error: type mismatch;
 found   : Boolean(true)
 required: Int
       "hej" * true
```

A type error means the compiler cannot make the types match in e.g. a function call. In Scala we find out about type errors already at compilation, whereas in other languages (e.g. Javascript) such errors are discovered during execution, possibly leading to hard-to-find bugs that may first manifest much later.

**Solution Task 5.** *Literals and types.*

a)

| | | | | |
|---|---|---|---|---|
| `1` | 1 | $\rightsquigarrow$ | E | `Int` |
| `1L` | 2 | $\rightsquigarrow$ | G | `Long` |
| `1.0` | 3 | $\rightsquigarrow$ | J | `Double` |
| `1D` | 4 | $\rightsquigarrow$ | F | `Double` |
| `1F` | 5 | $\rightsquigarrow$ | H | `Float` |
| `'1'` | 6 | $\rightsquigarrow$ | I | `Char` |
| `"1"` | 7 | $\rightsquigarrow$ | A | `String` |
| **`true`** | 8 | $\rightsquigarrow$ | C | `Boolean` |
| **`false`** | 9 | $\rightsquigarrow$ | B | `Boolean` |
| `()` | 10 | $\rightsquigarrow$ | D | `Unit` |

b) The value goes beyond the limit of what fits in a 32-bit integer and ”wraps around” to the smallest possible integer `Int.MinValue` because that is how binary arithmetic with limited bits works in the CPU.

```
scala> Int.MaxValue + 1
res3: Int = -2147483648

scala> Int.MinValue
res4: Int = -2147483648
```

c) Both are integers but `Long` can represent larger numbers than `Int`.

d) Both are floating-point types but `Double` has double precision and can represent larger numbers with more decimal digits.

**Solution Task 6.** *Mathematical functions. Using documentation.*

a) Computing $2^{64} - 1$ using `math.pow` as below yields approximately $1.8 \cdot 10^{19}$

```
scala> math.pow(2, 64) - 1
res0: Double = 1.8446744073709552E19
```

b) Yes, the return type annotation `: Double` can be omitted.

- Why can the return type be omitted?
  Because the compiler's type inference can deduce the return type.
- Why might one want to omit it?
  It is shorter to write without it.
- Reasons to specify the return type:
  - With explicit return type you get help from the compiler, already at compile time, to check that the expression to the right of the equals sign is of the expected type.
  - By explicitly specifying the return type, readers who only see the method signature (and not the implementation) can clearly see what is returned.

c) About 500 $km$.

```
scala> circumference(12750 / 2) / 80
res0: Double = 500.6913291658733
```

**Solution Task 7.** *Variables and assignment. Mutable and immutable variables.*

a)

| | | | |
|---|---|---|---|
| *After line 1:* | a: Int 13 | | |
| *After line 2:* | a: Int 13 | b: Int 14 | |
| *After line 3:* | a: Int 13 | b: Int 14 | c: Double 54.0 |
| *After line 4:* | a: Int 13 | b: Int 14 | c: Double 54.0 |
| *After line 5:* | a: Int 0 | b: Int 14 | c: Double 54.0 |
| *After line 6:* | a: Int 0 | b: Int 14 | c: Double 55.0 |

b) Immutable variables are declared with the keyword **val**. It is not possible to assign a new value to an immutable variable; attempting to do so results in a compile-time error that says **error: reassignment to val**. Compile-time errors can be recognized by the text **error:**, as shown below:

```
scala> b = 0
<console>:12: error: reassignment to val
       b = 0
         ^
```

**Solution Task 8.** *Random numbers with* `math.random()`.

a) From documentation:

```
/** Returns a Double value with a positive sign,
 *  greater than or equal to 0.0 and less than 1.0.
 */
def random(): Double
```

Documentation comments, which start with `/**` and end with `*/`, give a description of how the function works. After the doc comment comes the function header, which here tells us the function is named `random` and always returns a `Double`. (The tool `scaladoc` can use documentation comments to automatically generate websites with special documentation pages and search functions.)

b)

```
scala> def roll: Int = (math.random() * 6 + 1).toInt

scala> roll
res0: Int = 4

scala> roll
res1: Int = 1
```

**Solution Task 9.** *Repetition with* **for**, `foreach` *and* **while**.

a)

```
for i <- 1 to 100 do print(s"$roll, ")
```

b)

```
(1 to 100).foreach(i => print(s"$roll, "))
```

c)

```
var i = 1
while i <= 100 do { print(s"$roll, "); i = i + 1 }
```

```
var i = 1
while i <= 100 do
    print(s"$roll, ")
    i += 1
```

**Solution Task 10.** *Alternatives with* **`if`** *statement and* **`if`** *expression.*

a)

```
for i <- 1 to 100 do
  if roll == 6 then print("CONGRATULATIONS! ") else print(":(")
```

or

```
for (i <- 1 to 100) if (roll == 6) print("CONGRATULATIONS! ") else print(":(")
```

b)

```
var i = 1
var n = 0
while i <= 100 do
  if roll == 6 then n = n + 1
  i = i + 1
println("The number of sixes: " + n)
```

c)

```
def isAdult(age: Int) = age >= 18
```

**Solution Task 11.** *Sequence, statement and block.*

a) The statements create this output:

```
true!hej
true!hej
true!hej
true!hej
```

b)

```
scala> def p = { print("hej"); print("true"); println("!")}
scala> p;p;p;p
```

c)

- Curly braces are used to group multiple statements. Curly braces are needed if you want to define a function consisting of more than one statement. Since Scala 3 you can alternatively use indentation to define a multi-line function with multiple statements.

- Semicolons separate multiple statements. Semicolons are only needed if you want to write multiple statements on the same line.

**Solution Task 12.** *Integer division.*

| | | | | |
|---|---|---|---|---|
| `4 / 42` | 1 | ⇝ | F | `0: Int` |
| `42.0 / 2` | 2 | ⇝ | C | `21.0: Double` |
| `42 / 4` | 3 | ⇝ | B | `10: Int` |
| `42 % 4` | 4 | ⇝ | G | `2: Int` |
| `4 % 42` | 5 | ⇝ | A | `4: Int` |
| `40 % 4 == 0` | 6 | ⇝ | D | `true : Boolean` |
| `42 % 4 == 0` | 7 | ⇝ | E | `false: Boolean` |

**Solution Task 13.** *Boolean values.*

a) `true`
b) `false`
c) `true`
d) `true`
e) `false`
f) `true`
g) `true`
h) `true`
i) Exception thrown: `java.lang.ArithmeticException: / by zero`
j) `false`

**Solution Task 14.** *Boolean variables.*

2: Nothing is printed.

4: `watch out!!!`

**Solution Task 15.** *Turtle graphics with Kojo.*

a) Starting your Kojo program with `erase` causes execution to start in the same initial state: a blank canvas where the turtle points upward, the pen is down, and the pen color is red. That makes reasoning about what the program does from start to finish easier compared to if execution depends on results of previous executions.

b)

```
erase

front; left
front; left
front; left
front; left
```

c)

```
erase

front; left
front; right
```

```
front; left
front; right

front; left
front; right

front; left
```

## 1.2 Extra Tasks; additional practice

**Solution Task 16.** *Value and type.*

| | | | | |
|---|---|---|---|---|
| `1.0 + 18` | 1 | $\rightsquigarrow$ | H | `19.0: Double` |
| `(41 + 1).toDouble` | 2 | $\rightsquigarrow$ | K | `42.0: Double` |
| `1.042e42 + 1` | 3 | $\rightsquigarrow$ | A | `1.042E42: Double` |
| `12E6.toLong` | 4 | $\rightsquigarrow$ | I | `12000000: Long` |
| `32.toChar.toString` | 5 | $\rightsquigarrow$ | E | `" ": String` |
| `'A'.toInt` | 6 | $\rightsquigarrow$ | B | `65: Int` |
| `0.toInt` | 7 | $\rightsquigarrow$ | F | `0: Int` |
| `'0'.toInt` | 8 | $\rightsquigarrow$ | D | `48: Int` |
| `'9'.toInt` | 9 | $\rightsquigarrow$ | L | `57: Int` |
| `'A' + '0'` | 10 | $\rightsquigarrow$ | C | `113: Int` |
| `('A' + '0').toChar` | 11 | $\rightsquigarrow$ | J | `'q': Char` |
| `"*!%#".charAt(0)` | 12 | $\rightsquigarrow$ | G | `'*': Char` |

**Solution Task 17.** *Statements and expressions.*

a) An expression can be evaluated and yields a useful value. A statement *does* something (e.g. prints something), but does not yield a useful value.

b) `println()`

c)
`valueMissing` contains Unit
Prints Unit
Prints `"()"`
Prints `"()"`
Prints first 'hej' from the inner call then `()` from the outer call

d) `Unit`

e) `Unit`

**Solution Task 18.** *Procedure with parameter.*

a)

```
var highscore = 0
```

b)

```
def updateHighscore(points: Int): Unit =
  if points > highscore then
    highscore = points
    println("REKORD!")
  else println("DON'T GIVE UP!")
```

c)

```
def updateHighscore(points: Int): String =
```

```
  if points > highscore then
    highscore = points
    "REKORD!"
  else "DON'T GIVE UP!"
```

**Solution Task 19.** *Floating point arithmetic.*

a)

```
scala> Double.MinPositiveValue
res0: Double = 4.9E-324
```

b)

```
scala> Double.MaxValue + Double.MinPositiveValue == Double.MaxValue
res2: Boolean = true
```

**Solution Task 20.** ***`if`*** *statements.*

1. Output: **`false`**
2. Output: **`true`**
3. Nothing is printed, the function is declared but not executed.
4. Output: `1:kop 2:munt 3:kop 4:kop 5:munt` or similar depending on the random values from `math.random()`.

**Solution Task 21.** ***`if`*** *expressions.* Note the type `Any` on the last two expressions.

```
scala> if vegetable == "tomato" then "good" else "not good"
res0: String = not good

scala> if fruit == "banana" then "good" else "not good"
res1: String = good

scala> if true then vegetable else 42
res2: Any = cucumber

scala> if false then vegetable else 42
res3: Any = 42
```

**Solution Task 22.** *Modulo operator `%` and Boolean values.*

a)

```
scala> def isEven(n: Int): Boolean = n % 2 == 0

scala> isEven(42)
res0: Boolean = true

scala> isEven(43)
res1: Boolean = false
```

b)

```
scala> def isOdd(n: Int): Boolean = !isEven(n)

scala> isOdd(42)
res2: Boolean = false

scala> isOdd(43)
res3: Boolean = true
```

**Solution Task 23.** *Differences between* ***var***, ***val***, ***def***.

a)

```
  scala> var x = 30
  x: Int = 30

  scala> x + 1
  res6: Int = 31

  scala> x = x + 1
  x: Int = 31

  scala> x == x + 1
  res7: Boolean = false

  scala> val y = 20
  y: Int = 20

  scala> y = y + 1
  <console>:12: error: reassignment to val
         y = y + 1
           ^

  scala> var z = { println("hej z!"); math.random() }
  hej z!
  z: Double = 0.3381365875903367

  scala> def w = { println("hej w!"); math.random() }
  w: Double

  scala> z
  res8: Double = 0.3381365875903367

  scala> z
  res9: Double = 0.3381365875903367

  scala> z = z + 1
  z: Double = 1.3381365875903368

  scala> w
  hej w!
  res10: Double = 0.06420209879434557

  scala> w
  hej w!
  res11: Double = 0.5777951341051852
```

```

scala> w = w + 1
<console>:12: error: value w_= is not a member of object
       w = w + 1
```

b)

- `var name = expression` is used to declare a mutable variable. The name can be reassigned to new values using assignment statements.
- `val name = expression` is used to declare an immutable variable which cannot be changed after initialization. Attempting to assign it a new value results in a compile-time error.
- `def name = expression` is used to declare a function whose expression is evaluated every time it is called.

**Solution Task 24.** *Difference between* ***`if`*** *and* ***`while`***.

- Line 3: If lucky (50% chance) you get a single win printed once.
- Line 4: If lucky you get many wins in a row. The probability of $n$ wins in a row is $(\frac{1}{2})^n$.

## 1.3 Advanced Tasks; challenges

**Solution Task 25.** *Logic and De Morgan's Laws.*

a) `point > 1000`

b) `point > 100`

c) `point <= highscore`

d) `point <= 0 || point >= highscore`

e) `point >= 0 && point <= highscore`

f) `clear`

g) `!clear`

**Solution Task 26.** *The string interpolator* `s`.

a)

```
The name 'Kim Finkodare' has 12 letters.
```

b)

```
println(s"$f has ${f.size} letters.")
println(s"$e has ${e.size} letters.")
```

**Solution Task 27.** *Assignment operators.*

| | | | | | | |
|---|---|---|---|---|---|---|
| *After line1:* | a: | Int | 40 | | | |
| *After line2:* | a: | Int | 40 | b: | Int | 80 |
| *After line3:* | a: | Int | 50 | b: | Int | 80 |
| *After line4:* | a: | Int | 50 | b: | Int | 70 |
| *After line5:* | a: | Int | 100 | b: | Int | 70 |
| *After line6:* | a: | Int | 100 | b: | Int | 35 |

**Solution Task 28.** *Big numbers.*

a) `BigInt` can be used instead of `Int` for very large integers. There is also `Long` which has double the range of `Int`, while `BigInt` can have an arbitrarily large number of digits (until memory runs out) and thus represent enormously large numbers. `BigDecimal` can be used instead of `Double` for very large decimal numbers.

b)

```
scala> BigInt(2).pow(64)
res0: scala.math.BigInt = 18446744073709551616
```

c) Calculations are much slower and they are somewhat trickier to use.

**Solution Task 29.** *Precedence rules*

a) `77:  Int`

b) `13: Int`

c) `-13: Int`

**Solution Task 30.** *Documentation of packages in Java and Scala.*

a) Scala: `Pi`, Java: `PI`

b) You can search for and filter all occurrences of a particular string.

c) Computes the hypotenuse (Pythagorean theorem) without risking rounding errors in intermediate calculations.

**Solution Task 31.** *Accuracy and exceptions in arithmetic expressions.*

a) `-2147483648` which corresponds to `Int.MinValue`.

b) An exception is thrown: `java.lang.ArithmeticException: / by zero`

c) `1.0000000000000001E8` (as expected)

d) Rounded to `1E9` (floating-point arithmetic accuracy issue: a big float plus a (too) small float can give the same number; the small number 'disappears').

e) `45.00000000000001` (floating-point arithmetic accuracy issue: mathematically should be exactly 45.)

f) `Infinity` (same as `Double.PositiveInfinity`, representing positive infinity).

g) `2147483647` which is `Int.MaxValue`.

h) `NaN` meaning 'Not a Number'.

i) `NaN` meaning 'Not a Number'.

j) An exception is thrown: `java.lang.Exception: PANG!!!`

**Solution Task 32.** *Modulo arithmetic with negative numbers.* In Scala the result has the same sign as the dividend.

```
scala> 1 % 2
res0: Int = 1

scala> -1 % 2
res1: Int = -1

scala> -1 % -2
res2: Int = -1

scala> 1 % -2
res3: Int = 1
```

**Solution Task 33.** *Literal identifiers.*

a) The variable is named 'literal val', backticks allow names otherwise illegal, e.g. containing spaces or keywords.

b) Backticks in Scala allow all kinds of characters in names. Example use: Java has a method `java.lang.Thread.yield` but in Scala `yield` is a keyword; to work around this in Scala you can write `java.lang.Thread.`yield`

**Solution Task 34.** *`java.lang.Integer`, hexadecimal literals, BigDecimal.*

a)

```
scala> import Integer.{toBinaryString => toBin, toHexString => toHex}

scala> for i <- Seq(33, 42, 64) do println(s"$i \t ${toBin(i)} \t ${toHex(i)}")
33   100001    21
42   101010    2a
64   1000000   40
```

b) The hexadecimal integer $10c$ can be written as literal `0x10c` in Scala, Java and many other languages: [1]

```
scala> 0x10c
res0: Int = 268
```

c) [2]

```
scala> val c = 299792458
c: Int = 299792458

scala> BigDecimal(0x10).pow(c)
res68: scala.math.BigDecimal = 2.1248929632279066130609861108876 72E+360986089
```

**Solution Task 35.** *String formatting.*

```
val str = f"The giant cucumber is $g%1.3f meter lang"
```

(If you think `$g%1.3f` looks cryptic, take comfort that you now carry on a venerable heritage from the ancient language C and the legendary function `printf` to coming generations of initiated code wizards.)

**Solution Task 36.** *Multiplication warning.*

a) The second multiplication overflows the maximum allowed value of an `Int`. In the third multiplication an exception `java.lang.ArithmeticException: integer overflow` is thrown instead.

```
scala> Math.multiplyExact(1, 2)
res70: Int = 2

scala> Int.MaxValue * 2
res71: Int = -2

scala> Math.multiplyExact(Int.MaxValue, 2)
java.lang.ArithmeticException: integer overflow
  at java.lang.Math.multiplyExact(Math.java:867)
  ... 42 elided
```

[1] https://en.wikipedia.org/wiki/0x10c
[2] https://c418.bandcamp.com/album/0x10c

b) Used when you want to be 100

**Solution Task 37.** *Extra operators for exact multiplication.*

a)

```
scala> Int.MaxValue *! 1
res0: Int = 2147483647

scala> Int.MaxValue *! 2
java.lang.ArithmeticException: integer overflow
  at java.lang.Math.multiplyExact(Math.java:867)
  at IntExtra.$times$bang(<console>:16)
  ... 32 elided
```

b)

```
extension (i: Int)
  def *!(j: Int) = Math.multiplyExact(i,j)
  def +!(j: Int) = Math.addExact(i,j)
  def -!(j: Int) = Math.subtractExact(i,j)
```

c) It quickly becomes very cryptic with names consisting of multiple special characters. If you *really* want such operators it is *very* appropriate to also provide clear-text variants:

```
extension (i: Int)
  def mulExact(j: Int) = Math.multiplyExact(i,j)
  def *!(j: Int) = i mulExact j

  def addExact(j: Int) = Math.addExact(i,j)
  def +!(j: Int) = i addExact j

  def subExact(j: Int) = Math.subtractExact(i,j)
  def -!(j: Int) = i subExact j
```

# 2. Solution Excercise02

## 2.1 Basic Tasks

**Solution Task 1.** *Match concepts with descriptions.*

| | | | | |
|---|---|---|---|---|
| compile | 1 | ⇝ | C | machine code is generated from one or more source files |
| script | 2 | ⇝ | G | single code file, no main program needed |
| object | 3 | ⇝ | M | groups variables and functions |
| @main | 4 | ⇝ | E | where execution of the compiled program starts |
| program argument | 5 | ⇝ | L | can be passed via the args parameter to main |
| data structure | 6 | ⇝ | J | many elements in a whole; element-wise access |
| collection | 7 | ⇝ | K | data structure with elements of the same type |
| sequence collection | 8 | ⇝ | F | data structure with elements in a certain order |
| Array | 9 | ⇝ | D | a mutable, indexable sequence collection |
| Vector | 10 | ⇝ | B | an immutable, indexable sequence collection |
| Range | 11 | ⇝ | N | a collection representing a range of integers |
| yield | 12 | ⇝ | A | used in for-expressions to create a new collection |
| algorithm | 13 | ⇝ | H | step-by-step description of a solution to a problem |
| implementation | 14 | ⇝ | I | a specific realization of an algorithm |

**Solution Task 2.** *Using the terminal.*

a)

```
> mkdir hello
> cd hello
> pwd
```

b)

```
> cd ..
> ls
```

**Solution Task 3.** *Create and run a Scala script.*

a)

```
Sum of the first 1000 numbers is: 500500
```

b) The compilation error is: `')' expected, but eof found`

c) The file should look like this:

```
val n = args(0).toInt
val summa = (1 to n).sum
println(s"Sum of the first $n numbers is: $summa")
```

The output is:

```
Sum of the first 5001 numbers is: 12507501
```

d) The runtime error is:

```
java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0
```

Since the array `args` is empty when no program arguments are given, the index 0 does not exist.

**Solution Task 4.** *Scala application with `@main`.*

a) The compiler created 5 files in subdirectories under `.scala-build` named:

```
'hello$package.class'  'hello$package$.class'  'hello$package.tasty'
run.class   run.tasty
```

b) You get the error message if you remove the last curly brace. `eof` in the error message means end-of-file. This error message is common for unmatched parentheses, but the compiler often has difficulty giving a good error message if one of the parentheses is missing and it may point to an incorrect line for where it expects the missing character.

c) `Syntax Error: Expected a toplevel definition`. Without curly braces, indentation determines which parts of the code belong together. If you remove the indentation on the last line with the print statement, the compiler interprets this as being *outside* the main function and you get an error message because it is not allowed to have statements by themselves at top level. (However, it is allowed to have standalone statements in a script with `.sc` at the end of the filename.)

d) The annotation `@main` tells the compiler that the function is a main program and can serve as an entry point for execution.

Under the hood, the compiler creates an object with the same name as your main program. In that object, the compiler generates a method called `main` which takes a string array as a parameter and returns `Unit`. A compiled program must have at least one object with exactly one such main method because the JVM execution environment expects it and calls such a main method with a string array containing any program arguments when execution starts.

An alternative to `@main` is to define a so-called *primitive* main method in a single object. (This was necessary in old Scala 2, before the simpler `@main` annotation arrived in Scala 3.)

```
object Hello:
  def main(args: Array[String]): Unit =
    val message = "Hello world!"
    println(message)
```

**Solution Task 5.** *Create and use collections.*

| | | | | |
|---|---|---|---|---|
| `val xs = Vector(2)` | 1 | ⇝ | I | new reference to a sequence of length 1 |
| `val ys = Array.fill(9)(0)` | 2 | ⇝ | C | new reference to a mutable sequence |
| `Vector.fill(9)(' ')` | 3 | ⇝ | J | new immutable sequence with blank characters |
| `xs(0)` | 4 | ⇝ | E | shortcut for `apply(0)` |
| `xs.apply(0)` | 5 | ⇝ | F | indexing, returns the first element |
| `xs :+ 0` | 6 | ⇝ | A | new collection with a zero appended at the end |
| `0 +: xs` | 7 | ⇝ | H | new collection with a zero prepended at the beginnin |
| `ys.mkString` | 8 | ⇝ | K | new string with all elements concatenated |
| `ys.mkString(",")` | 9 | ⇝ | G | new string with commas between elements |
| `xs.map(_.toString)` | 10 | ⇝ | D | new collection, elements converted to strings |
| `xs.map(_.toInt)` | 11 | ⇝ | B | new collection, elements converted to integers |

**Solution Task 6.** *Compare `Array` and `Vector`.*

a)

| | | | | |
|---|---|---|---|---|
| Vector | 1 | ⇝ | B | immutable |
| Array | 2 | ⇝ | A | mutable |

b)

| | | | | |
|---|---|---|---|---|
| Vector | 1 | ⇝ | B | creates variants with more/other elements quickly from existing ones |
| Array | 2 | ⇝ | A | slow when changing size (requires copying everything) |

c)

| | | | | |
|---|---|---|---|---|
| Vector | 1 | ⇝ | A | `xs == ys` is **`true`** if all elements are equal |
| Array | 2 | ⇝ | B | unlike other Scala collections, == does not check content equality |

**Solution Task 7.** *Calculate sum, min and max in `args`.*

```
@main def sumMinMax(args: Int*): Unit =
  println(s"${args.sum} ${args.min} ${args.max}")
```

```
> scala run sum-min-max.scala -- hej
Illegal command line: java.lang.NumberFormatException: For input string: "hej"
```

**Solution Task 8.** *Algorithm: SWAP.*

a) The pseudocode may look like this:

```
Declare integer variable temp.
Copy value from x to temp.
```

```
Copy value from y to x.
Copy value from temp to y.
```

b)

You need to declare a temporary variable where you can save one of the values so that it is not overwritten in the first assignment.

```
val temp = x
x = y
y = temp
```

**Solution Task 9.** *Indexing and assignment in Array with SWAP.*

```
@main def swapFirstLastArg(args: String*): Unit =
  val xs = args.toArray
  if xs.length > 1 then
    val temp = xs(0)
    xs(0) = xs(xs.length - 1)
    xs(xs.length - 1) = temp
  println(xs.mkString(" "))
```

**Solution Task 10.** ***for*** *expressions and* `map` *expressions.*

| | | | |
|---|---|---|---|
| `for x <- xs yield x * 2` | 1 | $\rightsquigarrow$ A | `Vector(2, 4, 6)` |
| `for i <- xs.indices yield i` | 2 | $\rightsquigarrow$ E | `Vector(0, 1, 2)` |
| `xs.map(x => x + 1)` | 3 | $\rightsquigarrow$ D | `Vector(2, 3, 4)` |
| `for i <- 0 to 1 yield xs(i)` | 4 | $\rightsquigarrow$ B | `Vector(1, 2)` |
| `(1 to 3).map(i => i)` | 5 | $\rightsquigarrow$ C | `Vector(1, 2, 3)` |
| `(1 until 3).map(i => xs(i))` | 6 | $\rightsquigarrow$ F | `Vector(2, 3)` |

**Solution Task 11.** *Algorithm: SUMBUG*

a) Bug: Since `i` is never incremented, the program gets stuck in an infinite loop. Fix: Add a statement at the end of the while-block that increases the value of i by 1. Bug: Since sum is only increased by 1 each time, the result will be the sum of n ones, not the first n integers. Fix: Change so that the sum increases by `i` each iteration, instead of 1. For -1, the result is 0. Explanation: i starts at 1 and is thus never less than n which is -1. The while block executes zero times, and after sum receives its initial value it never changes.

b) The sum is 39502716.
One possible implementation:

```
@main def sumn(n: Int): Unit =
  var sum = 0
  var i = 1
  while i <= n do
    sum = sum + i
    i = i + 1
```

```
  println(sum)
```

## 2.2 Extra Tasks; additional practice

**Solution Task 12.** *Algorithm: MAXBUG*

a) Bug: `i` is never incremented. The program gets stuck in an infinite loop. Fix: Add a statement that increments i at the end of the while loop.

b) Here is one implementation:

```
@main def maxn(args: String*): Unit =
  var max = Int.MinValue
  val n = args.length
  var i = 0
  while i < n do
    val x = args(i).toInt
    if x > max then
      max = x
    i += 1
  println(max)
```

c) Change the line initializing max to `var max = args(0).toInt`

d) To avoid `java.lang.IndexOutOfBoundsException: 0` you need a check that ensures nothing is done if the collection `args` is empty:

```
@main def maxn(args: String*): Unit =
  if args.size > 0 then
    var max = args(0).toInt
    val n = args.size
    var i = 0
    while i < n do
      val x = args(i).toInt
      if x > max then
        max = x
      i += 1
    println(max)
  else
    println("Empty")
```

**Solution Task 13.** *Algorithm MIN-INDEX.*

a) An unnecessary comparison occurs, but does not affect the result.

b)

```
def indexOfMin(xs: Array[Int]): Int =
  var minPos = 0
  var i = 1
  while i < xs.size do
    if xs(i) < xs(minPos) then
      minPos = i
    i += 1
  if xs.size > 0 then minPos else -1
```

**Solution Task 14.** *The data structure Range.*

a) value: `Range(1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

b) value: `Range(1,2,3,4,5,6,7,8,9,10)`
type: `scala.collection.immutable.Range`

c) value: `Range(0,5,10,15,20,25,30,35,40,45)`
type: `scala.collection.immutable.Range`

d) value: `10`, type: `Int`

e) value: `Range(0,5,10,15,20,25,30,35,40,45,50)`
type: `scala.collection.immutable.Range`

f) value: `11`, type: `Int`

g) value: `Range(0,1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

h) value: `Range(0,1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

i) value: `Range(0,1,2,3,4,5,6,7,8,9)`
type: `scala.collection.immutable.Range`

j) value: `Range(0,1,2,3,4,5,6,7,8,9,10)`
type: `scala.collection.immutable.Range.Inclusive`

k) value: `Range(0,1,2,3,4,5,6,7,8,9,10)`
type: `scala.collection.immutable.Range.Inclusive`

l) value: `Range(0,5,10,15,20,25,30,35,40,45)`
type: `scala.collection.immutable.Range`

m) value: `Range(0,5,10,15,20,25,30,35,40,45,50)`
type: `scala.collection.immutable.Range`

n) value: `11`, type: `Int`

o) value: `500500`, type: `Int`

## 2.3 Advanced Tasks; challenges

**Solution Task 15.** *Rock-Paper-Scissors game.* One (easy-to-understand?) solution that tries all combinations:

```
def winner(user: Int, computer: Int): String =
  if      choices(user) == "Rock" && choices(computer) == "Paper" then "Computer"
  else if choices(user) == "Rock" && choices(computer) == "Scissors"  then "You"
  else if choices(user) == "Paper" && choices(computer) == "Rock" then "You"
  else if choices(user) == "Paper" && choices(computer) == "Scissors"  then "Computer"
  else if choices(user) == "Scissors"  && choices(computer) == "Rock" then "Computer"
  else if choices(user) == "Scissors"  && choices(computer) == "Paper" then "You"
  else "None"
```

A trickier solution (and harder to understand?) using modulo arithmetic:

```
def winner(user: Int, computer: Int): String =
  val result = (user - computer + 3) % 3
  if user == computer then "None"
  else if result == 1 then "You"
  else "Computer"
```

Modulo arithmetic requires the elements in `choices` to be in *loses-to* order, that is Rock, Paper, Scissors. Adding 3 is to avoid negative numbers which behave differently in modulo arithmetic.

**Solution Task 16.** *Compare execution time for resizing between `Array` and `Vector`.*

a) With a computer that has an `i7-4790K CPU @ 4.00GHz` this was the result:

```
scala> def time(block: => Unit): Double =
     |   val t = System.nanoTime
     |   block
     |   (System.nanoTime - t)/1e6  // gives milliseconds
def time(block: => Unit): Double

scala> val as = Array.fill(1e6.toInt)(0)
val as: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ...
large output truncated, print value to show all

scala> val vs = Vector.fill(1e6.toInt)(0)
val vs: Vector[Int] = Vector(0, 0, 0, 0, 0, ...
large output truncated, print value to show all

scala> val ast = (for i <- 1 to 10 yield time(as :+ 0)).sum / 10.0
val ast: Double = 1.8719819999999998

scala> val vst = (for i <- 1 to 10 yield time(vs :+ 0)).sum / 10.0
val vst: Double = 0.006485099999999999

scala> ast / vst
val res3: Double = 288.6589258453995
```

b) `Vector` is two orders of magnitude faster in this example. The reason is that every size change of an `Array` requires allocation and element-wise copying of a completely

new `Array`, whereas the immutable `Vector` can reuse the entire data structure with already allocated elements when new elements are added.

**Solution Task 17.** *Memory usage for `Range`.*

a) The variable `interval` refers to objects taking up 12 bytes.

b) The variable `sequence` refers to objects taking up about 4 billion bytes.

**Solution Task 18.** *Examine the generated byte-code.*

a) The function `plusxy` looks like this:

```
public int plusxy(int, int);
  descriptor: (II)I
  flags: (0x0001) ACC_PUBLIC
  Code:
    stack=2, locals=3, args_size=3
       0: iload_1
       1: iload_2
       2: iadd
       3: ireturn
    LineNumberTable:
      line 2: 0
    LocalVariableTable:
      Start  Length  Slot  Name   Signature
          0       4     0  this   Lplusxy$package$;
          0       4     1     x   I
          0       4     2     y   I
```

The instruction `iadd` performs the actual addition.

b) There is one more parameter in the bytecode. The instruction `iadd` now occurs twice. The `iadd` instruction adds exactly two integers at a time.

```
public int plusxyz(int, int, int);
  descriptor: (III)I
  flags: (0x0001) ACC_PUBLIC
  Code:
    stack=2, locals=4, args_size=4
       0: iload_1
       1: iload_2
       2: iadd
       3: iload_3
       4: iadd
       5: ireturn
    LineNumberTable:
      line 2: 0
    LocalVariableTable:
      Start  Length  Slot  Name   Signature
          0       6     0  this   Lplusxyz$package$;
          0       6     1     x   I
          0       6     2     y   I
          0       6     3     z   I
```

c) The prefix `i` in the instruction name `iadd` stands for "integer" and indicates that integer addition is intended.

# 3. Solution `Excercise03`

**Solution Task 1.** *Match concepts with description.*

| | | | | |
|---|---|---|---|---|
| function header | 1 | ⇝ | C | has parameter list and optionally a return type |
| function body | 2 | ⇝ | N | code executed on function call |
| parameter list | 3 | ⇝ | A | describes names and types of parameters |
| block | 4 | ⇝ | O | may have local names; last line gives value |
| named arguments | 5 | ⇝ | H | arguments can be passed in any order |
| default argument | 6 | ⇝ | L | arguments can be omitted |
| value call | 7 | ⇝ | B | argument evaluated before call |
| name call | 8 | ⇝ | D | delayed evaluation of argument |
| map | 9 | ⇝ | F | applies a function to each element of a collection |
| pure function | 10 | ⇝ | I | always returns same result for same arguments |
| predicate | 11 | ⇝ | G | a function returning a boolean value |
| random seed | 12 | ⇝ | M | produces a repeatable pseudo-random sequence |
| anonymous function | 13 | ⇝ | E | function without a name, also called lambda |
| recursive function | 14 | ⇝ | K | a function that calls itself |
| stack trace | 15 | ⇝ | J | list of calls at runtime error |

**Solution Task 2.** *Define and call functions.*

a)

```
def increase(x: Int = 1): Int = x + 1
```

b) `5`

c)

```
def reduce(x: Int = 1): Int = x - 1
```

d) `1`

e)

- *Short, simplified explanation:* The parameter in the function header is a local name for input that can be used in the function body, while the argument is the actual value of the parameter that is passed in the call.

- *Longer, more precise explanation:* A **parameter** is a declaration of an immutable variable in a function header whose name is locally available in the function body. At a call, the parameter name is *bound* to a specific argument. An **argument** is an expression that is applied to a function at call time. Normally the argument is evaluated before the call happens, but if the parameter type is preceded by `=>` the evaluation of the argument is delayed and instead happens *each time* the parameter name occurs in the function body.

**Solution Task 3.** *Implement function in different ways.*

a)

```
def sumFirst(n: Int): Int = ???
```

b)

```
def sumFirst(n: Int): Int = (1 to n).sum
```

```
scala> sumFirst(-1)
val res0: Int = 0
```

c)

```
def sumFirst(n: Int): Int =
  var result = 0
  var i = 1
  while i <= n do
    result += i
    i += 1
  end while
  result
end sumFirst
```

```
scala> sumFirst(-1)
val res1: Int = 0
```

**Solution Task 4.** *The text game AliensOnEarth.*

a) `"penguin"` is the best option with probability $\frac{1}{2} + \frac{1}{2} \cdot \frac{1}{3} = \frac{2}{3}$

b)

| | | | | |
|---|---|---|---|---|
| `options.indices` | 1 | $\rightsquigarrow$ | B | integer sequence with all indices in a sequence |
| `"1X2".toLowercase` | 2 | $\rightsquigarrow$ | A | convert string to lowercase |
| `Random.nextInt(n)` | 3 | $\rightsquigarrow$ | C | random number in range `0 until n` |
| `try { } catch { }` | 4 | $\rightsquigarrow$ | E | catch exceptions to prevent crash |
| `""" ... """` | 5 | $\rightsquigarrow$ | F | string spanning multiple lines |
| `s.stripMargin` | 6 | $\rightsquigarrow$ | D | removes margin up to vertical bar |
| `e.printStackTrace` | 7 | $\rightsquigarrow$ | G | prints exception information |

**Solution Task 5.** *Pure functions.*

- The functions `inc`, `addY` and `isPalindrome` are pure. Note that the variable `y` is initialized to `0` and cannot be changed because it is declared with the keyword `val`.

**Solution Task 6.** *Apply function to each element in a collection. Function as argument.*

| | | | |
|---|---|---|---|
| `for i <- 1 to 3 yield increment(i)` | 1 | ⇝ E | `Vector(2, 3, 4)` |
| `Vector(2, 3, 4).map(i => increment(i))` | 2 | ⇝ A | `xs` |
| `xs.map(increment)` | 3 | ⇝ B | `Vector(4, 5, 6)` |
| `xs.map(increment).map(increment)` | 4 | ⇝ D | `Vector(5, 6, 7)` |
| `xs.foreach(increment)` | 5 | ⇝ C | `()` |

**Solution Task 7.** *Anonymous functions.*

| | | | |
|---|---|---|---|
| `(0 to 2).map(i => i + 1)` | 1 | ⇝ B | `(2 to 4).map(i => i - 1)` |
| `(1 to 3).map(_ + 1)` | 2 | ⇝ D | `Vector(2, 3, 4)` |
| `(2 to 4).map(math.pow(2, _))` | 3 | ⇝ E | `Vector(4.0, 8.0, 16.0)` |
| `(3 to 5).map(math.pow(_, 2))` | 4 | ⇝ A | `Vector(9.0, 16.0, 25.0)` |
| `(4 to 6).map(_.toDouble).map(_ / 2)` | 5 | ⇝ C | `Vector(2.0, 2.5, 3.0)` |

**Solution Task 8.** *Create your own control structure using call-by-name parameter.*

a)

```
def repeat(n: Int)(block: => Unit): Unit =
  var i = 0
  while i < n do
    block
    i += 1
  end while
```

b)

```
repeat(100):
  val diceRoll = (math.random() * 6 + 1).toInt
  print(s"\$diceRoll ")
```

c) If the parameter `block` were not declared with call-by-name, the argument would be evaluated once before the call and then the same result used at every iteration. With call-by-name, the block can contain code that e.g. updates a variable which we want to happen every iteration. Call-by-name is similar to the argument code being "pasted" into each place in the function body where the parameter name occurs.

**Solution Task 9.** *Learn to read a stack trace.*A stack trace contains the following information:

1. an error message
2. names of all functions called at the time of the runtime error, according to all activation records on the call stack
3. current namespace for each function, that is package/singleton object

4. source file name for each function
5. line number in each function
6. the first function shown is the one where the error occurred
7. possibly the error occurs in standard library functions and then your own function is earlier in the call chain

Example of a stack trace:

```
> cat fel.scala
@main def run =
  println("Hej Scala!" + Vector().head)
> scala run fel.scala
Compiling project (Scala 3.3.0, JVM)
Compiled project (Scala 3.3.0, JVM)
Exception in thread "main" java.util.NoSuchElementException: empty.head
  at scala.collection.immutable.Vector.head(Vector.scala:279)
  at fel$package$.run(fel.scala:2)
  at run.main(fel.scala:1)
>
```

## 3.1 Extra Tasks; additional practice

**Solution Task 10.** *Function with multiple parameters.*

a)

```
def sum(x: Int, y: Int): Int = x + y

def diff(x: Int, y: Int): Int = x - y
```

b) It is `-100` because `0 - 100 == -100`

c) It is 15 because the nested call corresponds to
`diff(100, 42 + 43) == (100 - 85)`

d) It is 185 because the nested call corresponds to
`sum(42 + 43, 100 - 0) == (85 + 100)`

e) It is 256 because `Byte.MaxValue == 127` and `Byte.MinValue == -128` and `sum(127 + 128, 1)`

**Solution Task 11.** *Average value.*

```
def avg(x: Int, y: Int): Double = (x + y) / 2.0
```

**Solution Task 12.** *Function calls with named arguments.*

a)

```
Namn: Triangelsson, Stina
Namn: Oval, Viktor
```

b)

- The caller can choose the order themselves.

- The code becomes easier to understand if parameter names are self-describing.
- Helps to prevent bugs caused by swapped parameters.

**Solution Task 13.** *Function as a genuine value.*

a)

| | | | |
|---|---|---|---|
| `multipleCalls(1, greet)` | 1 | ⇝ D | `f2("Hello!")` |
| `multipleCalls(3, greet)` | 2 | ⇝ B | `multipleCalls(3, f1)` |
| `multipleCalls(2, f1)` | 3 | ⇝ A | `f2("Hello!\nHello!")` |
| `multipleCalls(1, f3)` | 4 | ⇝ C | `f3()` |

b) `f1` and `f3` have type `() => Unit` and `f2` has type `String => Unit`.

c) No. `f1` and `f2` have two different function types.

d) Yes, that works fine.

e) No. When the function has no parameter, the compiler needs more information to be sure that you want a function value.

f) Yes! Now with type information in place, the compiler knows what you want to do.

**Solution Task 14.** *Discarded result values and return type `Unit`.*

a) Procedures return the empty value and `println` is a procedure. When the empty value is printed, you see `()`.

b) Procedures return the empty value. If you explicitly specify return type `Unit`, you have a better chance that the compiler can warn you when calculations are discarded. A warning does not stop execution, but is a way for the compiler to give you tips about things that might need fixing in your code.

c) In Scala variable declaration, just like an assignment statement, is not an expression and has no value.

d) The code becomes easier to read and the compiler gets better possibility to help with warnings if result values risk being discarded.

**Solution Task 15.** *Call-by-name.*

The block is an expression that has the value `(): Unit`. Evaluation of the block happens where the name `b` appears in the procedure body, which is twice.

```
scala> doThisTwice { println("goededag") }
goededag
goededag
```

## 3.2 Advanced Tasks; challenges

**Solution Task 16.** *Mutability of parameters.*

a) No, in Scala the parameter is immutable and it will be a compile error if you try to assign it a new value in the function body.

b) c) Yes, it is perfectly fine in both Java and Python to change the value of the parameter in the function body by assignment, but the code risks to become confusing. https://stackoverflow.com/questions/2970984

**Solution Task 17.** *Call-by-value and call-by-name.*

a) Each call of `snark` causes a print and a delay before returning 42.
`42 + 42 == 84` which becomes the expression value.

```
scala> snark + snark
snark snark val res1: Int = 84
```

b) The expression `snark` is evaluated directly at the call and the parameter `x` is bound to the value 42 and in the function body $42+42$ is computed. The print happens only once.

```
callByValue(snark)
snark val res2: Int = 84
```

c) The evaluation of the expression `snark` is delayed until each occurrence of parameter `x` in the function body. The print happens twice.

```
callByName(snark)
snark snark val res3: Int = 84
```

d) Evaluation of the expression `zzz` is delayed until each occurrence of parameter `x` in the function body. The print happens once because **`val`** variables are assigned their value once at the delayed initialization.

```
callByName(zzz)
snark val res4: Int = 84
```

**Solution Task 18.** *Create your own control structure for iteration with loop variable.*

a)

```
def repeat(n: Int)(p: Int => Unit): Unit =
  var i = 0
  while i < n do
    p(i)
    i += 1
  end while
end repeat
```

b)

```
repeat(100){ i =>
  print("i ")
```

```
  println(math.random())
}
```

You can use fewer braces with the help of colon:

```
repeat(100): i =>
  print("i ")
  println(math.random())
```

**Solution Task 19.** *Split parameter list and curried functions.*

a)

```
1 scala> def add2(a: Int)(b: Int) = a + b
2 def add2(a: Int)(b: Int): Int
3
4 scala> add2(1)(1)
5 val res0: Int = 2
```

b)

- Line 3:

```
doremi doremi doremi
```

- Line 5:

```
lalalalalalala
```

**Solution Task 20.** *Recursion.*

a) `countdown` prints out `x` and does a recursive call with `x - 1` as argument, but only if the base condition `x > 0` is true. The result is a finite repetition. `finalCountdown` calls itself recursively but lacks a base case that can stop the recursion, which generates an infinite repetition. At -128 there is *overflow* since the bits are not enough for larger negative numbers and the counting starts over at 127. (If you reduce the delay to `Thread.sleep(1)` it becomes quite fast *stack overflow*)

b) Since `1/x` was *after* the recursive call in the previous subtask, we never reached this (potentially disastrous) calculation, but just put many activation frames on the stack at each call. If we place `1/x` *before* the recursive call, we reach this expression immediately and an exception is thrown due to division by zero.

c) The last line leads to many more recursive calls, as the base condition and recursion are constructed. Feel free to insert a `println` statement before the recursive call and examine exactly what happens.

**Solution Task 21.** *Investigate tail recursion by throwing exceptions.* `countdown` is tail-recursive because the recursive call is *last* and can be optimized to a `while`-loop by the compiler. It is fine to run all the way until it explodes, even with 10000 calls, and the error message only contains one call to `countdown`.

`countdown2` is not tail-recursive because it has an expression `after` the recursive call. In the error printout you can see all recursive calls to `countdown2` before the base case occurs. At `countdown2(10000)` the base case is not fulfilled before a `StackOverflowError` occurs.

**Solution Task 22.** *`@tailrec` annotation.* The first time `countNoTailrec(100000L)` is called it causes `StackOverflowError`. With annotation `@tailrec` we get a compilation error because the compiler cannot optimize a non-tail-recursive function. If the function is rewritten, the compiler can optimize it so that the recursion is replaced with a **`while`**-loop and we can run as long as we want without stack overflow. And it runs very fast!!

# 4. Solution `Excercise04`

## 4.1 Basic Tasks; preparation for the lab

**Solution Task 1.** *Match concepts with descriptions.*

| | | | | |
|---|---|---|---|---|
| module | 1 | ⇝ | C | code unit with abstractions that can be reused |
| singleton object | 2 | ⇝ | B | module that can have state; exists in a single instance |
| package | 3 | ⇝ | D | module creating namespace; machine code gets own folder |
| import | 4 | ⇝ | F | makes name locally available without full path |
| export | 5 | ⇝ | P | makes name visible outside as a member of this object |
| lazy initialization | 6 | ⇝ | G | allocation occurs only when name is referenced |
| member | 7 | ⇝ | E | belongs to an object; accessed via dot notation if visible |
| attribute | 8 | ⇝ | H | variable representing (part of) an object's state |
| method | 9 | ⇝ | A | function that is a member of an object |
| private | 10 | ⇝ | K | modifies visibility of an object member |
| overloading | 11 | ⇝ | J | methods with same name but different parameter types |
| name shadowing | 12 | ⇝ | L | local name hides same name in enclosing block |
| namespace | 13 | ⇝ | I | environment where all names are unique |
| uniform access | 14 | ⇝ | M | changing between def and val does not affect usage |
| dot notation | 15 | ⇝ | O | used to access non-private parts |
| type alias | 16 | ⇝ | N | alternative name for type, often improves readability |

**Solution Task 2.** *Nested singleton objects, import, visibility and dot notation.*

a)

```
object Underworld:
  var x = 0
  var y = 1

  object Mole:
    var x = Underworld.x + 10
    var y = Underworld.y + 9

  object Worm:
    private var x = Mole.x
    var y = Mole.y + 190
    def isMoleFood: Boolean = x == Mole.x && y == Mole.y
```

b)

```
scala> :load Underworld.scala
scala> import Underworld.*
scala> Worm.isMoleFood
val res0: Boolean = false
scala> Worm.y = Mole.y
scala> Worm.isMoleFood
```

```
val res1: Boolean = true
```

c)

```
scala> import Mole.*
scala> import Worm.*
scala> x = -1
scala> Mole.x
val res2: Int = -1

scala> Worm.x
1 |Worm.x
  |^^^^^^^^
  |variable x cannot be accessed as a member of Underwold.Worm.type from module

scala> Underworld.x
val res3: Int = 0
```

*Explanation:* When importing all visible members in Worm, those that are not private will shadow other members with the same name. It is Mole's `x` variable that is assigned `-1` because Worm's `x` is private and not visible externally. Underworld's members are shadowed by Worm's `y` and Mole's `x`, but you can still access them using dot notation.

**Solution Task 3.** *Export.*

a) Similarity: Both **`import`** and **`export`** control visibility. Difference: **`import`** controls local visibility *inside* an object while **`export`** controls visibility *outside* an object.

b) You can conveniently gather members from other objects and make them visible from your own object using **`export`**.

```
object MyObject:
  export java.awt.Color.*  // all colors become members of MyObject
  export math.{atan2, Pi}  // atan2 and Pi become members of MyObject
```

```
scala> object MyObject:
     |   export java.awt.Color.*
     |   export math.{atan2, Pi}
     |

scala> MyObject.RED
val res0: java.awt.Color = java.awt.Color[r=255,g=0,b=0]

scala> MyObject.atan2(3,3) / MyObject.Pi
val res1: Double = 0.25
```

**Solution Task 4.** *Tuples.*

a) `depth` has type `Double`.

b) `secret` has type `(String, (Int, Int, Double))`.

c)

```
object Underworld3D:
  private val secret = ("the ascent to the uberworld", (3, 4, 0.0))

  object Mole:
```

```
    var pos = (5, 3, math.random() * 10 + 1)

    def depth: Double  = pos._3

  object Worm:
    private var pos = (0, 0, 10.0)

    def isMoleFood: Boolean = pos == Mole.pos

    def isRightUnderTheRise: Boolean =
      pos._1 == secret._2._1 && pos._2 == secret._2._2
```

d) Zero-tuple.

**Solution Task 5.** *Lazy initialization.*

a) `"nu!"` is printed only the first time `z` is used.

```
scala> z
nu!
val res19: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)

scala> z
val res20: Array[Int] = Array(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
```

b) The array allocation happens the first time `z` is used (and not at declaration).

```
scala> lazy val z = { println("nu!"); Array.fill(1e9.toInt)(0)}
val z: Array[Int] = <lazy>

scala> z
nu!
java.lang.OutOfMemoryError: Java heap space
```

c) The print `"nu!"` occurs only when singleton object `zzz` is used for the first time. We should move the initialization of `b` before `a` or make `a` a **`lazy val`**.

d)

```
scala> import test.*
import test.*

scala> zzz.a      // "nu!" is printed when we use zzz
nu!               // this did *not* happen when we imported test
val res0: Int = 42

scala> buggig.a   // a is 0 because b was not initialized
val res1: Int = 0

scala> funkar.a   // with lazy val we avoid the problem
val res2: Int = 42


scala> zzz.a     // second time initialization already done, no "nu!"
val res3: Int = 42
```

e) `lazy val a = expression` means the initialization expression is evaluated *once*, but the evaluation is delayed until the name `a` is actually used. Whereas `def b = expression` means the function body expression is evaluated *each time* the name `b` is (possibly) used.

**Solution Task 6.** *Extension methods.*

a)

```
scala> extension (i: Int) def inc = i + 1
```

b)

```
scala> extension (i: Int) def dec = i - 1
```

c)

```
extension (i: Int)
  def inc = math.incrementExact(i)
  def dec = math.decrementExact(i)
```

d) Using `math.incrementExact` and `math.decrementExact` throws an exception if we cross the boundary:

```
scala> math.incrementExact(Int.MaxValue)
java.lang.ArithmeticException: integer overflow
  at java.base/java.lang.Math.incrementExact(Math.java:1023)
  at scala.math.incrementExact(package.scala:418)
  ... 34 elided
```

**Solution Task 7.** *Extension methods.*

a) According to the documentation, the `PixelWindow` class has these parameters:

- `width : Int` specifies the window width, default `800`
- `height: Int` specifies the window height, default `640`
- `title : String` specifies the window title, default `"PixelWindow"`
- `background: Color` specifies background color, default `java.awt.Color.black`
- `foreground: Color` specifies foreground color, default `java.awt.Color.green`

You can create new window instances like this:

```
val w1 = new introprog.PixelWindow()
val w2 = new introprog.PixelWindow(100, 200, "My nice new window")
```

b) You can also download the latest `introprog` like this:

```
curl -o introprog_3-1.4.0.jar -sLO https://fileadmin.cs.lth.se/introprog.jar
```

c)

```
> scala repl --jar introprog_3-1.4.0.jar
scala> val w = new introprog.PixelWindow(400,300,"HEJ")
scala> w.line(100, 100, 200, 100)
scala> w.line(200, 100, 200, 200)
scala> w.line(200, 200, 100, 200)
scala> w.line(100, 200, 100, 100)
```

d)

```
package hello

object Main:
  val w = new introprog.PixelWindow(400, 300, "HEJ")

  var color = java.awt.Color.red

  def square(p: (Int, Int))(side: Int): Unit =
    if side > 0 then
      // side == 1 gives a square that is a single pixel
      val d = side - 1

      w.line(p._1,     p._2,     p._1 + d, p._2,     color)
      w.line(p._1 + d, p._2,     p._1 + d, p._2 + d, color)
      w.line(p._1 + d, p._2 + d, p._1,     p._2 + d, color)
      w.line(p._1,     p._2 + d, p._1,     p._2,     color)

  def main(args: Array[String]): Unit =
    println("Draw square:")
    square(300,100)(50)
```

e)

```
> scala run hello-window.scala --jar introprog_3-1.4.0.jar --main-class hello.M
```

f)

```
> scala run hello-window.scala --dep se.lth.cs::introprog:1.4.0 --main-class he
```

g)

```
//> using scala 3.3
//> using dep se.lth.cs::introprog:1.4.0
```

**Solution Task 8.** *Color*.

a)

```
object Color:
  import java.awt.{Color as JColor}

  val mole   = new JColor( 51,  51,   0)
  val soil   = new JColor(153, 102,  51)
  val tunnel = new JColor(204, 153, 102)
```

b)

```
package hello

object Color:
  import java.awt.{Color as JColor}
```

```
  val mole   = new JColor( 51, 51,    0)
  val soil   = new JColor(153, 102, 51)
  val tunnel = new JColor(204, 153, 102)


object Main:
  val w = new introprog.PixelWindow(width = 400, height = 300, title = "HEJ")

  type Pt = (Int, Int)

  var color = java.awt.Color.red

  def straight(p:  Pt)(d: Int) = w.line(p._1, p._2, p._1 + d - 1, p._2, color)

  def fill(p: Pt)(s: Int) = for i <- 0 until s do straight((p._1, p._2 + i))(s)

  def square(p: (Int, Int))(side: Int): Unit =
    if (side > 0) then
      val d = side - 1  // side == 1 should give a square that is one pixel big
      w.line(p._1,     p._2,     p._1 + d, p._2,     color)
      w.line(p._1 + d, p._2,     p._1 + d, p._2 + d, color)
      w.line(p._1 + d, p._2 + d, p._1,     p._2 + d, color)
      w.line(p._1,     p._2 + d, p._1,     p._2,     color)

  def main(args: Array[String]): Unit =
    import Color.*
    color = soil
    fill(100,100)(75)
    color = tunnel
    fill(100,100)(50)
    color = mole
    fill(150,150)(25)
```

c) Tuple parentheses can be omitted in calls to `straight` and `fill` when a single tuple argument is the only parameter in its parameter list.

**Solution Task 9.** *Events.*

a) The immutable integer variable `KeyPressed` in `introprog.PixelWindow.Event` has the value 1.

b) The code line below handles the key press case:

```
case PixelWindow.Event.KeyPressed => println(s"lastKey == \$w.lastKey")
```

c) When the up arrow key on the keyboard is pressed, `w.lastKey` gets the string value `"Up"`. The test program prints the following when the up arrow key is pressed and released:

```
lastEventType: 1 => KeyPressed
lastKey == Up
lastEventType: 2 => KeyReleased
lastKey == Up
```

d) A loop that lets the user draw lines with the mouse:

```
var start = (0,0)
while w.lastEventType != PixelWindow.Event.WindowClosed do
  w.awaitEvent(10)  // wait for next event max 10 milliseconds
  w.lastEventType match {
    case PixelWindow.Event.MousePressed  =>
      start = w.lastMousePos

    case PixelWindow.Event.MouseReleased =>
      w.line(start._1, start._2, w.lastMousePos._1, w.lastMousePos._2)

    case PixelWindow.Event.WindowClosed  =>
       println("Goodbye!");
    case _ =>
  }
  PixelWindow.delay(100) // wait for 0.1 seconds
```

## 4.2 Extra Tasks; additional practice

**Solution Task 10.** *Functions are objects with an `apply` method.* Yes, you can write:

```
scala> plus(42, 43)
```

The compiler inserts `.apply` for you.

**Solution Task 11.** *Create modules using singleton objects.*

a)

```
scala> "pearIceCream".split('I')
val res0: Array[String] = Array(pear,ceCream )
```

b)

```
scala> Test()
--- FREQUENCY ANALYSIS OF:
Five ants are more than four elephants. Eat cucumber.
# letters : 43
# words   : 9
# sentences: 2

--- FREQUENCY ANALYSIS OF:
Galaxies in my breams. Tomato is good. Pear split.
# letters : 39
# words   : 9
# sentences: 3

--- FREQUENCY ANALYSIS OF:
Five ants are more than four elephants. Eat cucumber. Galaxies in my breams. To
# letters : 82
# words   : 18
# sentences: 5
```

c) The object `statistics` has mutable state in the variable `history`. The state changes when calling `printFreq`.

d)

```
object count:
  extension (s: String)
    def nbrOfLetters:Int = s.count(_.isLetter)
    def nbrOfWords:Int = split.words(s).size
    def nbrOfSentences: Int = split.sentences(s).size
```

**Solution Task 12.** *Tuples as parameters.*

```
def distxy(x1: Int, y1: Int, x2: Int, y2: Int): Double =
  hypot(x1 - x2, y1 - y2)

def distpt(p1: (Int, Int), p2: (Int, Int)): Double =
  hypot(p1._1 - p2._1, p1._2 - p2._2)

def distp(p1: (Int, Int))(p2: (Int, Int)): Double =
```

```
  hypot(p1._1 - p2._1, p1._2 - p2._2)
```

**Solution Task 13.** *Tuples as function results.*

```
def statistics(xs: Vector[Double]): (Int, Double, (Double, Double)) =
  (xs.size, xs.sum / xs.size, (xs.min, xs.max))
```

```
scala> statistics(Vector(0, 2.5, 5))
val res10: (Int, Double, (Double, Double)) = (3,2.5,(0.0,5.0))
```

**Solution Task 14.** *Create modules using packages.*

a)

```
> code paket.scala
> scala paket.scala
> find . -type d         # linux command that lists all subdirectories
./.scala-build/project_103be31561-3d0d386400/classes
./.scala-build/project_103be31561-3d0d386400/classes/main
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p2
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p2/
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p1
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p1/
./.scala-build/project_103be31561-3d0d386400/classes/main/gurka/tomat/banan/p1/
```

b)

```
> scala run paket.scala --main-class gurka.tomat.banan.Main
Hej paket p1.p11!
Hej paket p1.p12!
Hej paket p2.p21!
```

c) Yes, in Scala 3 packages are allowed to have variables and functions at top level. https://stackoverflow.com/a/56566166

## 4.3 Advanced Tasks; challenges

**Solution Task 15.** *How to do without* `do while` *in Scala 3?*

a) It gives a compilation error:

```
> scala repl --scala 3
Welcome to Scala 3.1.3 (17.0.3, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> var i = 0
var i: Int = 0

scala> do i += 1 while (i < 10)
-- [E103] Syntax Error: ----------------------------------------
1 |do i += 1 while (i < 10)
```

```
  |^^
  |Illegal start of statement
```

b)

```
> scala repl --scala 3
Welcome to Scala 3.1.3 (17.0.3, Java OpenJDK 64-Bit Server VM).
Type in expressions for evaluation. Or try :help.

scala> var i = 0
var i: Int = 0

scala> while
     |   i += 1
     |   i < 10
     | do ()

scala> i
val res0: Int = 10
```

**Solution Task 16.** *Postfix operators for increment and decrement.*

```scala
extension (i: Int)
  def ++ = i + 1
  def -- = i - 1
```

**Solution Task 17.** *Use ready package: Color chooser.*

a) The chosen color is returned after the user clicks **OK**

```
scala> introprog.Dialog.selectColor()
val res1: java.awt.Color = java.awt.Color[r=0,g=204,b=0]
```

b) The default color red is returned after the user clicks **Cancel**

c) Color chooser reverts to default color.

**Solution Task 18.** *Use ready package: user dialogs.*

a)

```
scala> introprog.Dialog.show("Game over!")
```

b) The function `input` returns a string which is the empty string `""` if the user clicks **Cancel**

```
scala> val name = introprog.Dialog.input("What is your name?")
name: String = Oddput Superkodare
```

c) The function `select` returns a string with the text on the button that the user clicked.

```
scala> introprog.Dialog.select("Wat kies jij?",Vector("Steen","Schaar","Papier"
val res4: String = Papier
```

**Solution Task 19.** *Create your own `jar` file.*

a)

`jar -create -verbose -file <name of created jar file> <name of what to package>`

b)

```
package hello

object Main:
  def main(args: Array[String]): Unit = println("Hello package!")
```

```
scala compile hello.scala --destination .
```

c)

```
> jar -c -v -f my.jar hello
> ls
> scala repl --jar my.jar
scala> hello.Main.main(Array())
Hello package!
```

d)

```
> scala run --jar my.jar  --main-class hello.Main
```

**Solution Task 20.** *How large is JDK8?* The JDK8 platform includes 4240 precompiled classes, organized in 217 different packages. See Stackoverflow: http://stackoverflow.com/questions/3112882

# 5. Solution Excercise05

## 5.1 Basic Tasks; preparation for the lab

**Solution Task 1.** *Match concepts with description.*

| | | | | |
|---|---|---|---|---|
| class | 1 | ⇝ | H | template for creating multiple instances of the same type |
| instance | 2 | ⇝ | I | copy of an object with its own state memory |
| constructor | 3 | ⇝ | M | creates instance, allocates space for state memory |
| class parameter | 4 | ⇝ | K | binds to arguments given at construction |
| reference equality | 5 | ⇝ | B | instances are considered different even if states are equal |
| structural equality | 6 | ⇝ | J | instances considered equal if they have the same state |
| case class | 7 | ⇝ | F | no need to write new; automatic structural equality |
| getter | 8 | ⇝ | L | indirect access to attribute value |
| setter | 9 | ⇝ | A | indirect assignment to attribute value |
| companion object | 10 | ⇝ | D | sees private members of class with same name |
| factory method | 11 | ⇝ | E | helper function for indirect construction |
| `null` | 12 | ⇝ | G | a value that does not refer to any instance |
| `new` | 13 | ⇝ | C | keyword for direct instance creation of a class |

**Solution Task 2.** *Class and instance.*

a)

| | | | | |
|---|---|---|---|---|
| `Singelpunkt.x` | 1 | ⇝ | B | `1` |
| `Punkt.x` | 2 | ⇝ | G | `value is not a member of object` |
| `val p  = new Singelpunkt` | 3 | ⇝ | C | `Not found: type` |
| `val p1 = new Punkt` | 4 | ⇝ | D | `p1: Punkt = Punkt@27a1a53c` |
| `val p2 = Punkt()` | 5 | ⇝ | F | `p2: Punkt = Punkt@51ab04bd` |
| `{ p1.x = 1; p2.x }` | 6 | ⇝ | E | `3` |
| `(new Punkt).y` | 7 | ⇝ | H | `2` |
| `{ val p: Punkt = null; p.x }` | 8 | ⇝ | A | `java.lang.NullPointerException` |

b)

| *error* | *type* | *explanation* |
|---|---|---|
| `value is not a member of object` | compile-time error | there is no instance named `Punkt`. *The error message refers to the fact that the class's auto-generated constructor proxy lacks a variable named* `x`. |
| `Not found: type` | compile-time error | there is no class named `Singelpunkt` |
| `NullPointerException` | runtime error | you cannot reference attributes in an instance that does not exist |

**Solution Task 3.** *Class parameters.*

a)

| | | | |
|---|---|---|---|
| `val p1 = Point(1, 2)` | 1 | ⇝ C | `p1: Point = Point@30ef773e` |
| `val p2 = Point()` | 2 | ⇝ A | `missing argument for parameter` |
| `val p2 = Point(3, 4)` | 3 | ⇝ E | `p2: Point = Point@218cf600` |
| `p2.x - p1.x` | 4 | ⇝ B | `2` |
| `Point(0, 1).y` | 5 | ⇝ F | `1` |
| `Point(0, 1, 2)` | 6 | ⇝ D | `too many arguments for constructor` |

b)

| *error* | *type* | *explanation* |
|---|---|---|
| `missing argument for parameter` | compile-time error | you must provide arguments at construction of class `Point` |
| `too many arguments for constructor` | compile-time error | the number of arguments does not correspond to the number of class parameters |

**Solution Task 4.** *Immutable class with default arguments.*

a)

| | | | |
|---|---|---|---|
| `val p1 = Point3D()` | 1 | ⇝ C | `p1: Point3D = Point3D@2eb37eee` |
| `val p2 = Point3D(y = 1)` | 2 | ⇝ F | `p2: Point3D = Point3D@65a9e8d7` |
| `Point3D(z = 2).z` | 3 | ⇝ E | `value cannot be accessed` |
| `p2.y = 0` | 4 | ⇝ B | `Reassignment to val` |
| `p2.y == 0` | 5 | ⇝ A | `false` |
| `p1.x == Point3D().x` | 6 | ⇝ D | `true` |

b) The problem is that as the class `Point3D` is declared it is not possible to read off the `z` coordinate after an instance is constructed. It would be better if the `z` attribute were also a `val`.

**Solution Task 5.** *Case class, `this`, equality, `toString` and companion object.*

a)

| | | | |
|---|---|---|---|
| `val p1 = Pt(1, 2)` | 1 | ⇝ E | `Pt(1,2)` |
| `val p2 = Pt(y = 3)` | 2 | ⇝ C | `Pt(0,3)` |
| `val p3 = MutablePt(5, 6)` | 3 | ⇝ A | `MPt(5,6)` |
| `val p4 = Mutable()` | 4 | ⇝ D | `Not found` |
| `p2.moved(dx = 1) == Pt(1, 3)` | 5 | ⇝ F | `true` |
| `p3.move(dy = 1) == MutablePt(5, 7)` | 6 | ⇝ B | `false` |

b) The compiler infers `MutablePt` because that is the type of the self-reference `this`.

```
scala> :type new MutablePt().move()
MutablePt
```

c) Instantiation with universal apply methods is sugar that makes code easier to read and write. This is possible thanks to the fact that at compilation automatically a constructor proxy is created that instantiates the object with the keyword **new**. A constructor proxy is a companion object with an apply method.

One case where **new** must explicitly be used is when implementing your own apply method in a companion object. If **new** is not used inside the apply method, the same method will be called recursively instead of creating a new instance. See the following example:

```
class Point3D(val x: Int, val y: Int, val z: Int)

object Point3D:
  var secretNumber = 42
  def apply(x: Int, y: Int, z: Int): Point3D =
    if secretNumber == 42 then
      Point3D(x, y, z) // The code will get stuck in an infinite loop.

    else new Point3D(x, y, z) // Works because 'new' is used.
```

d) A method that reads (parts of) an object's (private) state without modifying it is called a *getter*.

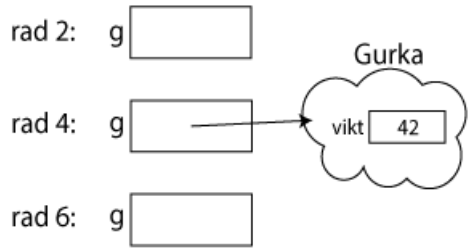
**Solution Task 6.** *Implement parts of the classes `Pos`, `KeyControl`, `Mole` and `BlockWindow` needed during the lab `blockbattle1`.*This task is lab preparation. Evaluate your solutions through your own tests in REPL.

a) It is not possible to call `Pos.moved(0,1)`. The reason is that `moved` does not exist in the companion object `Pos`, hence the error message "value `moved` is not a member of object `Pos`". To call a method defined inside a class you must call it through a (reference to an) instance of the class.

## 5.2 Extra Tasks; additional practice

**Solution Task 7.** *Instantiation with* **`new`** *and the value* **`null`**.

a) Lines 3 and 7 both give the error message `java.lang.NullPointerException`, because of attempts to reference members using a **`null`** reference, which does not point to any object.




b)

**Solution Task 8.** *Create a point class that can handle polar coordinates and a class that represents a polygon using these points.*

a)

```
package graphics

case class Point(x: Double, y: Double):
  val r: Double = math.hypot(x, y)
  val theta: Double = math.atan2(y, x)
  def +(p: Point): Point = Point(x + p.x, y + p.y)

object Point:
  def polar(r: Double, theta: Double): Point =
    Point(r * math.cos(theta), r * math.sin(theta))
```

b)

```
package graphics

case class Polygon(points: Vector[Point]):
  val nbrOfCorners: Int = points.length

object Polygon:
  def regular(nbrOfCorners: Int, radius: Double, midPoint: Point): Polygon =
    val points = new Array[Point](nbrOfCorners)
    for i <- 0 until nbrOfCorners do
      val theta = i * (2 * math.Pi) / nbrOfCorners
      points(i) = Point.polar(radius, theta) + midPoint
    end for
    Polygon(points.toVector)
```

c) A perfect circle cannot be created, but it is possible to get so close that it is impossible to see the corners. Try e.g. with 50 corners as below.

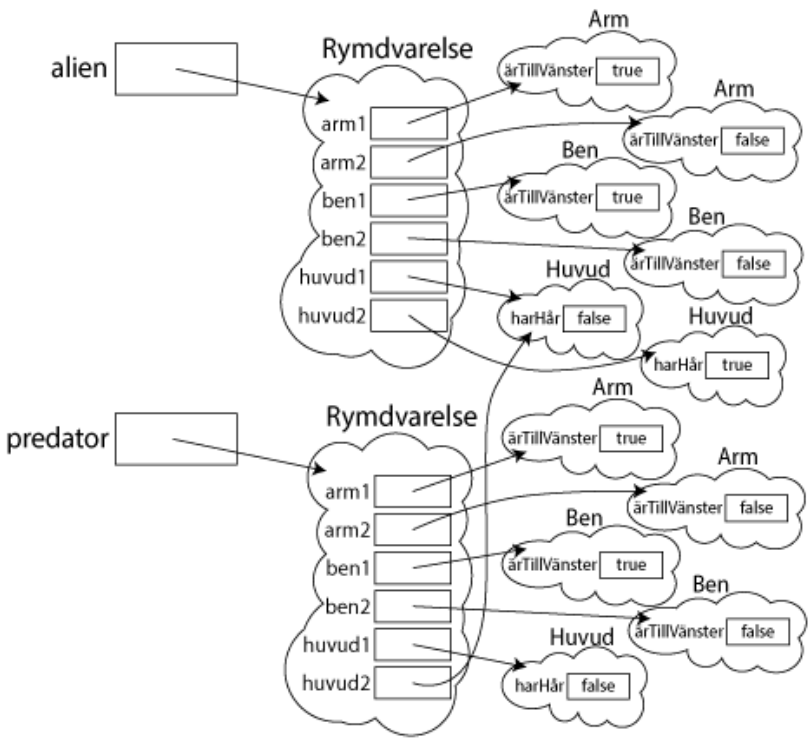
```
scala> import graphics.*
scala> val circle = Polygon.regular(50,70,Point(-25,-25))
scala> val window = PolygonWindow()
scala> window.draw(circle)
```

An irregular polygon can also be created. Use the constructor of `Polygon` directly. For example as below.

```
scala> val irregular =
         Polygon(Vector(Point(1,8), Point(33,14), Point(99,87), Point(42,56)))
scala> window.draw(irregular)
```

**Solution Task 9.** *Classes, instances and garbage.*

a) We create two aliens, `alien` and `predator`, each with two arms and two legs, and each with two heads (one bald and one hairy). Afterwards, neither `alien` nor `predator` is bald since both have one head with hair. Then the reference to `predator`'s hairy head is made to refer to `alien`'s bald head. Now the predator is totally bald and shares head with the alien.




b) Since there is no longer any reference pointing to that object, the garbage collector will take care of it and eventually the place in memory will be overwritten when the space is needed. Objects that have no reference to them cannot be accessed.

**Solution Task 10.** *Case class. Immutable square.*

a)

```
case class Square(val x: Int = 0, val y: Int = 0, val side: Int = 1):
  val area: Int = side * side

  def moved(dx: Int, dy: Int): Square = Square(x + dx, y + dy, side)

  def isEqualSizeAs(that: Square): Boolean = this.side == that.side
```

```
  def scale(factor: Double): Square =
    Square(x, y, (side * factor).round.toInt)

object Square:
  val unit: Square = Square()
```

b)

```
scala> val (s1, s2) = (Square(), Square(1, 10, 1))
val s1: Square = Square(0,0,1)
val s2: Square = Square(1,10,1)

scala> val s3 = s1 moved (1,-5)
val s3: Square = Square(1,-5,1)

scala> s1 isEqualSizeAs s3       // same size
val res0: Boolean = true

scala> s2 isEqualSizeAs s1       // same size
val res1: Boolean = true

scala> s1 isEqualSizeAs Square.unit   // s1 has side 1
val res2: Boolean = true

scala> s2.scale(math.Pi) isEqualSizeAs s2  // different size
val res3: Boolean = false

scala> s2.scale(math.Pi) == s2.scale(math.Pi) // same content
val res4: Boolean = true

scala> s2.scale(math.Pi) eq s2.scale(math.Pi)  // different objects
val res5: Boolean = false

scala> Square.unit eq Square.unit   // same object
val res6: Boolean = true
```

## 5.3 Advanced Tasks; challenges

**Solution Task 11.** *Content equality between different types.*

```
scala> 42 == "Fourtytwo"
1 |42 == "Fourtytwo"
  |^^^^^^^^^^^^^^^^^
  |Values of types Int and String cannot be compared with == or !=

scala> Cucumber(50) == Car("Sedan")
val res0: Boolean = false
```

The second expression is problematic because it will always result in **false**, since the classes `Cucumber` and `Car` are two incomparable types that should not be compared in terms of content equality. This reduces type safety which increases the risk of subtle bugs where wrong types are compared.

Equality comparisons done between primitive types are type-checked by the compiler and therefore can produce compilation errors if two different types, such as `Int` and `String`, are compared. However, this usually does not apply to user-defined types, which means that an equality comparison between different user-defined types will always result in **false**.

It is possible, however, to achieve the same type checks for user-defined types as for primitive types by importing `scala.language.strictEquality`.

```
import scala.language.strictEquality
class Cucumber(val weight: Int)

class Car(val type: String)
```

```
scala> Cucumber(50) == Car("Sedan")
1 |Cucumber(50) == Car("Sedan")
  |^^^^^^^^^^^^^^^^^^^^^^^^^^^
  |Values of types Cucumber and Car cannot be compared with == or !=
```

**Solution Task 12.** *Attribute representation. Private constructor. Factory method.*

a) It will be a compilation error, because the constructor is private.

```
scala> class Point private (val x: Int, val y: Int)
     | object Point:
     |   def apply(x: Int = 0, y: Int = 0): Point = new Point(x, y)
     |   val origo = apply()
     |
// defined class Point
// defined object Point

scala> new Point(0, 0)
1 |new Point(0, 0)
  |    ^^^^^
  |constructor Point cannot be accessed as a member of Point from module class
```

b)

- By having a private constructor and only allowing indirect instantiation via the factory method it is easy to change attribute representation in the future without existing code needing to change.
- The access rules for companion objects are such that companions see each other's private parts.

c)

```
class Point private (private val p: (Int, Int)):
  def x: Int = p._1
  def y: Int = p._2

object Point:
  def apply(x: Int = 0, y: Int = 0): Point = new Point(x, y)
  val origo = apply()
```

**Solution Task 13.** *Visibility of class parameters and constructor,* ***`private`****`[this]`*.

a) `Cucumber5` is broken. Since the weight in `Cucumber5` is private to the instance and not the class, an instance cannot access another instance's weight.

```
11 |  def buddyWeight = buddy.weight
   |                    ^^^^^^^^^^^
   |value weight cannot be accessed as a member of (Cucumber5.this.buddy : Cucu
```

b)

```
scala> new Cucumber1(42).weight
1 |new Cucumber1(42).weight
  |^^^^^^^^^^^^^^^^^^^
  |value weight cannot be accessed as a member of Cucumber1 from module class

scala> new Cucumber2(42).weight
val res0: Int = 42

scala> new Cucumber3(42).weight
1 |new Cucumber3(42).weight
  |^^^^^^^^^^^^^^^^^^^
  |value weight cannot be accessed as a member of Cucumber3 from module class

scala> val noneCucumber: Cucumber4 = null
val noneCucumber: Cucumber4 = null

scala> new Cucumber4(42, noneCucumber).buddyWeight
java.lang.NullPointerException: Cannot invoke "rs$line$1$Cucumber4.weight()" be
  at rs$line$1$Cucumber4.buddyWeight(rs$line$1:8)
  ... 38 elided

scala> new Cucumber4(42, new Cucumber4(84, null)).buddyWeight
val res2: Int = 84

scala> new Cucumber6(42)
1 |new Cucumber6(42)
  |     ^^^^^^
  |constructor Cucumber6 cannot be accessed as a member of Cucumber6 from modul
```

```

scala> new Cucumber7(-42)
1 |new Cucumber7(-42)
  |    ^^^^^^
  |constructor Cucumber7 cannot be accessed as a member of Cucumber7 from modul

scala> Cucumber7(-42)
java.lang.IllegalArgumentException: requirement failed: negative weight: -42

scala> val g = Cucumber7(42)
val g: Cucumber7 = Cucumber7@51fd1c7c

scala> g.weight
val res4: Int = 42

scala> g.weight = -1

scala> g.weight
val res5: Int = -1
```

**Solution Task 14.**

a)

Line 1:

```
  java.lang.IllegalArgumentException: requirement failed: negative weight: -42
```

`Cucumber8.apply` requires that `weight >= 0` otherwise `require` throws an exception.

Line 5:

```
  java.lang.IllegalArgumentException: requirement failed: negative weight: -1
```

The setter `weight_=` requires `weight >= 0` otherwise `require` throws an exception.

Line 7:

```
  java.lang.IllegalArgumentException: requirement failed: negative weight: -958
```

Since `42 - 1000` is less than zero `require` throws an exception.

b) You can impose your own specific requirements on what values are allowed so you have greater control that nothing unexpected happens.

**Solution Task 15.** *Object with mutable state.*

a)

```scala
class Frog private (initX: Int = 0, initY: Int = 0):
  private var _x: Int = initX
  private var _y: Int = initY
  private var _distanceJumped: Double = 0

  def x: Int = _x
  def y: Int = _y

  def jump(dx: Int, dy: Int): Unit =
    _x += dx
    _y += dy
```

```
    _distanceJumped += math.hypot(dx, dy)


  def randomJump: Unit =
    def rnd = util.Random.nextInt(10) + 1
    jump(rnd, rnd)

  def distanceToStart: Double = math.hypot(x,y)
  def distanceJumped: Double = _distanceJumped
  def distanceTo(f: Frog): Double = math.hypot(x - f.x, y - f.y)

object Frog:
  def spawn(): Frog = Frog()
```

b) Example test program:

```
object FrogTest:
  def test(): Unit =
    val f1 = Frog.spawn()
    assert(f1.x == 0 && f1.y == 0, "Test of spawn, reqt 1 & 4 failed.")

    f1.jump(4, 3)
    assert(f1.x == 4 && f1.y == 3, "Test of jump, reqt 1 & 4 failed.")

    f1.jump(4, 3)
    assert(f1.distanceJumped == 10, "Test of jump, reqt 2 failed.")

    f1.jump(-4, -3)
    assert(f1.distanceToStart == 5, "Test of jump, reqt 3 failed.")

    for x <- 1 to 10000 do
      val f2 = Frog.spawn()
      f2.randomJump
      assert(f2.x > 0 && f2.x <= 10 && f2.y > 0 && f2.y <= 10,
        "Test of randomJump, reqt 5 failed.")

    println("Test Ok!")
```

c) A method that is an indirect reading of attribute values is called a getter.

d)

```
class Frog private (initX: Int = 0, initY: Int = 0):
  private var _x: Int = initX
  private var _y: Int = initY
  private var _distanceJumped: Double = 0

  def jump(dx: Int, dy: Int): Unit =
    _x += dx
    _y += dy
    _distanceJumped += math.hypot(dx, dy)
```

```
  def x: Int = _x
  def x_=(newX: Int): Unit = // Setter for x
    _distanceJumped += math.abs(x - newX)
    _x = newX

  def y: Int = _y
  def y_=(newY: Int): Unit = // Setter for y
    _distanceJumped += math.abs(y - newY)
    _y = newY


  def randomJump: Unit =
    def rnd = util.Random.nextInt(10) + 1
    jump(rnd, rnd)

  def distanceToStart: Double = math.hypot(x,y)
  def distanceJumped: Double = _distanceJumped
  def distanceTo(f: Frog): Double = math.hypot(x - f.x, y - f.y)

object Frog:
  def spawn(): Frog = Frog()
```

e)

```
object FrogSimulation:
  def isAnyCollision(frogs: Vector[Frog]): Boolean =
    var found = false
    frogs.indices.foreach(i =>  // generate all pairs (i,j)
      for j <- i + 1 until frogs.size do
        if !found then
          found = frogs(i).distanceTo(frogs(j)) <= 0.5
    )
    found

  def jumpUntilCrash(n: Int = 100, initDist: Int = 8): (Int, Double) =
    val frogs = Vector.fill(n)(Frog.spawn())
    (0 until n).foreach(i => frogs(i).x = i * initDist)
    var count = 0
    while !isAnyCollision(frogs) do
      frogs(util.Random.nextInt(n)).randomJump
      count += 1
    (count, frogs.map(_.distanceJumped).sum)


  def run(nbrOfCrashTests: Int = 10) =
    for i <- 1 to nbrOfCrashTests do
      val (n, dist) = jumpUntilCrash()
      println(s"\nAntalet looprundor innan grodkrock: $n")
      println(s"Totalt avstånd hoppat av alla grodor: $dist")
```

**Solution Task 16.** *Object with mutable state.*

```
class Square private (val initX: Int, val initY: Int, val initSide: Int):
  private var nMoves = 0
  private var sumCost = 0.0

  private var _x = initX
  private var _y = initY

  private var _side = initSide

  private def addCost(): Unit =
    sumCost += math.hypot(x - initX, y - initY) * side

  def x: Int = _x
  def y: Int = _y

  def side = _side

  def scale(factor: Double): Unit = _side = (_side * factor).round.toInt

  def move(dx: Int, dy: Int): Unit =
    _x += dx; _y += dy
    nMoves += 1
    addCost()

  def moveTo(x: Int, y: Int): Unit =
    _x = x; _y = y
    nMoves += 1
    addCost()

  def cost: Double = sumCost

  def pay: Double = {val temp = sumCost; sumCost = 0; temp}

  override def toString: String =
    s"Square[($x, $y), side: $side, #moves: $nMoves times, cost: $sumCost]"

object Square:
  private var created = Vector[Square]()

  def apply(x: Int, y: Int, side: Int): Square =
    require(side >= 0, s"side must be positive: $side")
    val sq = (new Square(x, y, side))
    created :+= sq
    sq

  def apply(): Square = apply(0, 0, 1)

  def totalNumberOfMoves: Int = created.map(_.nMoves).sum

  def totalCost: Double = created.map(_.cost).sum
```

# 6. Solution Excercise06

## 6.1 Basic Tasks; preparation for the lab

**Solution Task 1.** *Match on constant values.*

a) Scala's **match** expression compares stepwise the value with each **case** and then returns a value belonging to the corresponding **case**.

b)

```
scala.MatchError
```

Runtime error, occurs for certain input during execution.

**Solution Task 2.** *Guards in case branches.*

The guard introduced at **case** 'c' generates a random number between 0 and 1 and if the number is not greater than 0.5, there will be no match with **case** 'c' and the program tries further until the default case.
The guard condition must be met in order to match as usual.

**Solution Task 3.** *Pattern match on attributes in case classes.*
G100true. When changing the order: Gtrue100.
**match** tests if the companion object Cucumber is of type Cucumber with two parameter values. The specified parameters are assigned names, weight is given the name w and isRotten the name rotten and then printed. If these names are switched, they are printed in the reversed order.

**Solution Task 4.** *Match on case objects and the usefulness of **sealed**.*

a)

```
Cannot extend sealed trait Color in a different source file
```

The error message occurs because the REPL treats each input individually and therefore does not allow the subtype Spades to extend the supertype Color since it was sealed. More about this later in the course...

b) -

c) Assuming **import** Deck._ has been written...

```
def paracolor(f: Color): Color = f match
  case Spades  => Clubs
  case Hearts => Diamonds
  case Diamonds   => Hearts
  case Clubs  => Spades
```

d)

```
<console>:17: warning: match may not be exhaustive.
It would fail on the following input: Diamonds
```

The warning appears already at compilation.

e)

```
scala.MatchError: Diamonds (of class Diamonds)
  at .paracolor(<console>:17)
```

This is a runtime error.

f) If a class is **sealed**, it means that if an element should be matched and is a subtype of this class, Scala gives a warning already at compile time if there is a risk of a `MatchError`, i.e., if the **match** expression is not exhaustive and there are cases not covered by a **case**.
A sealed supertype means that the programmer gets a warning at compile time if a case is not covered, and in that case which of the subtypes is missing, as well as other help from the compiler. This requires, however, that all the subtypes share the same file as the sealed class.

**Solution Task 5.** *Pattern match enumeration.*a)

```
def paracolor(f: Color): Color = f match
  case Color.Spades => Color.Clubs
  case Color.Hearts => Color.Diamonds
  case Color.Diamonds => Color.Hearts
  case Color.Clubs => Color.Spades
```

Similar to task **????** an **import** statement can also be written here to access members of `Color` without dot notation. However, it is not always advantageous to import members into the global namespace, since name clashes can occur. Suppose an example where we work on a program with a graphical user interface that has a color `Red` defined. Also suppose that we want to import yet another red color for the suits hearts and diamonds, also named `Red`. In this scenario, a name clash would occur since `Red` is already defined, so the import could not proceed.

b) When pattern matching, **sealed trait** used with **case** objects works in practice similarly to using **enum**. We saw in subtask **????** that the REPL warned already at compile time that the match is not exhaustive. This also applies when using **enum**.

**Solution Task 6.** *Meaning of lowercase and uppercase initial letters when matching.*

a) Both `str` and `whatever` match the input, regardless of what it is, because they start with a lowercase letter.
However, `str` has a guard that the string must start with `c`, which means only **val** `g = "cucumber"` matches this. However, **val** `x = "urka"` is caught by `whatever`, which lacks a guard.

b)

```
<console>:16: warning: patterns after a variable pattern cannot match (SLS 8.1.
```

and

```
<console>:17: warning: unreachable code due to variable pattern 'tomato' on lin
```

Even though a class `tomato` exists, Scala treats the **match** case as a catch-all because it starts with a lowercase letter. This means any object not of type `Cucumber` will print *tomato* and the last case cannot be reached.

c)

```
case `tomato` => println("tomato")
```

**Solution Task 7.** *Match on the contents of a Vector.*

```
hguod
hguod
42
```

For each element in `xss`, a match is done that results in a string. What happens in each branch is explained below.

1. The first match branch is never activated because `xss` contains no empty vector.
2. The second branch fits `Vector("hej")`, and the variable `a` is bound to `"hej"`.
3. The third branch matches `Vector("on", "dough")` where the first value is ignored (underscore) and the second value is bound to `b`.
4. The fourth branch matches a sequence with three values where the middle value is `"x"`. The last branch is not triggered here but would match anything not caught earlier.

**Solution Task 8.** *Use `Option` and match on potentially missing values.*

a)

1. `var maybe` becomes an `Option` holding `Int` but with no value, called `None`.
2. Since `var maybe` has no value, its size is 0.
3. `var maybe` is assigned the value 42 which is stored in a `Some` indicating a value exists.
4. Since `var maybe` now contains a value, its size is 1.
5. Since `var maybe` contains a value, it is not empty.
6. Since `var maybe` contains a value, it is defined.
7. `def increaseIfExists` matches an `Option[Int]` with its different cases.
   If there is a value, i.e., `opt: Option[Int]` is a `Some`, it returns a `Some` with the original value plus 1.
   If there is no value, i.e., `opt: Option[Int]` is a `None`, it returns `None`.
8. -
9. -
10. -
11. `def increaseIfExists` is applied to `maybe` and returns a `Some` with the value in `maybe` plus 1, i.e., 43.
12. `def öka` takes an `Int` and returns this value plus 1.
13. `map` applies `def öka` to the sole element in `maybe`, 42. This returns a `Some` with value 43 assigned to `merKanske`.

b)

1. `val` `sentence` becomes a `Some` with value 42.
2. `val` `notSentence` becomes an `Option[Int]` without value, i.e., `None`.
3. `map(_ + 1)` is applied on `sentence` and increases the existing value by 1 to 43.
4. `map(_ + 1)` is applied on `notSentence` but since no value exists, it remains `None`.
5. `map(_ + 1)` is applied again on `notSentence` but now `orElse` method is included. If an `Option` has no value (`None`), the code in `orElse` executes, which here prints out *missing* for the missing value.
6. The same call from the previous line is executed this time on `meningen` and since a value exists, only the first part that increases the value by 1 runs.

This method can be used instead of the `match`-version in the previous example due to its simpler form. An `Option` contains either a value or none so a longer `match` is unnecessary.

c)

1. A vector `xs` is created with every fifth number from 42 to 82.
2. An empty `Int` vector `e` is created.
3. `headOption` takes the first value of vector `xs` and returns it wrapped in an `Option`, `Some(42)`.
4. The first value in vector `xs` is wrapped in an `Option` and then extracted by `get`, 42.
5. Same as previous line but uses `getOrElse`, which returns 0 if the `Option` is `None`. Since `xs` has at least one value, this returns the same as before, 42.
6. Like previous line but instead of returning 0 if missing, returns an `Option[Int]` with 0 as value.
7. `headOption` tries to take the first element of empty vector `e` but since no value exists, returns `None`.
8. 
```
java.util.NoSuchElementException: None.get
```

   Like preceding line returns `None`, but calling `get` on `None` causes a runtime error.
9. Like previous returns `None` from empty vector `e`, but `getOrElse` returns 0 instead.
10. Like previous uses `getOrElse` on the `None` and returns an `Option[Int]` holding 0.
11. A vector containing `xs` and three `e` vectors is created.
12. `map` applies `lastOption` on each sub-vector extracting last elements into a new vector. Since `e` is empty, returns `None` for those.
13. Same as previous but uses `flatten` to remove all `None` elements leaving only actual values.

14. `lift` method fetches element at index 0 in `xs` wrapped in an `Option` which is `Some(42)`.
15. `lift` tries to get element at index 1000 in `xs`, which doesn't exist, so returns `None`.
16. Same as previous but applied to vector `e`, and then `getOrElse` returns 0 since `None`.
17. `find` searches `xs` for first element > 50 and returns it in `Some(52)`.
18. `find` searches `xs` for first element < 42 but none exists, so returns `None`.
19. `find` on `e` searching for element > 42 and prints *HITTAT!* if found. Since `e` is empty, `None` is returned and `foreach` does not run.

d) Using `-1` as a return value for missing or error cases can cause runtime errors that are hard to detect. **`null`** can cause crashes if misused during execution. `Option` avoids such problems, and if `getOrElse` or similar is used, the program does not crash. Moreover, functions returning `Option` do not need as much documentation about what can be returned; instead, the code itself shows it explicitly.
Finally, `Option` is more type-safe than **`null`**. When you return an `Option`, the type of the contained value is specified, if any, which makes it easier to understand and limits what can be returned.

**Solution Task 9.** *Throw exceptions.*

a)

1. An `Exception` is thrown with message *PANG!*.
2. Various types of `Exception` are shown.
3. An exception of type `IllegalArgumentException` is thrown with message *wrong wrong wrong*.
4. An exception with message `stormWind!` is thrown and caught by the **`catch`** expression. A **`match`** inspects the exception and prints the message, and returns -1.

b) For example:
`OutOfMemoryError`, when the program runs out of memory.
`IndexOutOfBoundsException`, when accessing a vector position larger than available.
`NullPointerException`, when calling a method or similar on a null reference.

c) If try and catch branches have the same type, here `Int`, the compiler infers that type for the entire expression. If the catch branch returns a different type such as `String`, then the most precise inferred type for the whole expression is `Matchable`, a direct subtype of the most general type `Any`.

**Solution Task 10.** *Catch exceptions with `scala.util.Try`.*

a)

1. **`def`** `pang` is defined that throws an `Exception` with message *PANG!*.
2. Scala utilities `Try`, `Success` and `Failure` are imported.

3. `def` `pang` is called in `Try` which catches the exception and wraps it in a `Failure`.
4. The method `recover` matches the exception in the `Failure` from above with a `case` and turns that `Failure` into a `Success`, similar to catch.
5. The string *silent* runs in the previous test but since no exception is thrown it is wrapped in a `Success` and `recover` does nothing. It only handles exceptions.
6. `def` `maybePang` is defined that is equally likely to return the string *silent* or call `pang`.
7. `def` `maybeOk` is defined that tests `def` `maybePang` with `Try`.
8. A vector `xs` is filled with results of 100 runs of `maybeOk` consisting of `Success` and `Failure`.
9. The element at index 13 in vector `xs` is matched to either `Success` (prints ":)") or `Failure` (prints ":( " plus error message).
10. -
11. -
12. Method `isSuccess` tests if element at `xs(13)` is `Success`, returns `true` if so.
13. Method `isFailure` tests if element at `xs(13)` is `Failure`, returns `true` if so.
14. Method `count` counts how many elements in `xs` are `Failure`.
15. Method `find` searches `xs` for a `Failure` element and returns it wrapped in an `Option`.
16. `badOpt` is assigned first `Failure` found in `xs`.
17. `goodOpt` is assigned first `Success` found in `xs`.
18. Printing `badOpt` yields `Option[scala.util.Try[String]] = Some(Failure(java.lang.Exception: PAN`
19. Calling `get` on `badOpt` returns the `Failure` inside the `Option`.
20. Calling `get` again on the `Failure` extracts and throws the exception.
21. Using `getOrElse` on the `Failure` returns the alternative string "bomb disarmed!" since it is a `Failure`.
22. Using `getOrElse` on the `Success` returns the contained string "silent".
23. The same method is mapped over all elements of `xs` and printed.
24. `toOption` is mapped over `xs` turning `Success` into `Some` with the string "silent" and `Failure` into `None`.
25. `flatten` removes `None`s from the vector, leaving just the `Some` elements.
26. The `size` returns the number of `Success` elements counted.

b) `pang` has return type `Nothing`, a special Scala type which is subtype of all types but has no values.

c) `Nothing` is a subtype of every type in Scala's hierarchy, including `String`, which implies `String` includes both strings and `Nothing`, so the inferred type is `String`.

## 6.2 Advanced Tasks; challenges

**Solution Task 11.** *Use matching or dynamic binding?*

a)

```
package vegopoly

trait Vegetable:
  def weight: Int
  def isRotten: Boolean
  def isWorthEating: Boolean

case class Cucumber(weight: Int, isRotten: Boolean) extends Vegetable:
  val isWorthEating: Boolean = (!isRotten && weight > 100)

case class Tomato(weight: Int, isRotten: Boolean) extends Vegetable:
  val isWorthEating: Boolean = (!isRotten && weight > 50)

object Main:
  def randomWeight: Int = (math.random()*500 + 100).toInt

  def randomDecay: Boolean = math.random() > 0.8

  def randomCucumber: Cucumber = Cucumber(randomWeight, randomDecay)

  def randomTomato: Tomato = Tomato(randomWeight, randomDecay)

  def randomVegetable: Vegetable =
    if math.random() > 0.2 then randomCucumber else randomTomato

  def main(args: Array[String]): Unit =
    val harvest = Vector.fill(args(0).toInt)(randomVegetable)
    val edible = harvest.filter(_.isWorthEating)
    println("Number of vegetables harvested: " + harvest.size)
    println("Number of edible vegetables: " + edible.size)
```

b) Add the following `case class`:

```
case class Broccoli(weight: Int, isRotten: Boolean) extends Vegetable:
  val isEatable: Boolean = (!isRotten && weight > 80)
```

Also add the following in `object Main` before `def randomVegetable`:

```
def randomBroccoli: Broccoli = Broccoli(randomWeight, randomRotten)
```

Finally, modify `def randomVegetable` to:

```
def randomVegetable: Vegetable =     // choose e.g. this distribution:
  val rnd = math.random()
  if rnd > 0.5 then randomCucumber      // 50% chance cucumber
```

```
else if rnd > 0.2 then randomTomato // 30% chance tomato
else randomBroccoli             // 20% chance broccoli
```

c) The advantages of the **match** version, and pattern matching generally, are that it is easy to change how matching is done. This makes it easy to change the definition of edibility. However, if these change rarely but vegetables change often, the polymorphic alternative may be preferred since it is easier to implement and change when vegetables change.

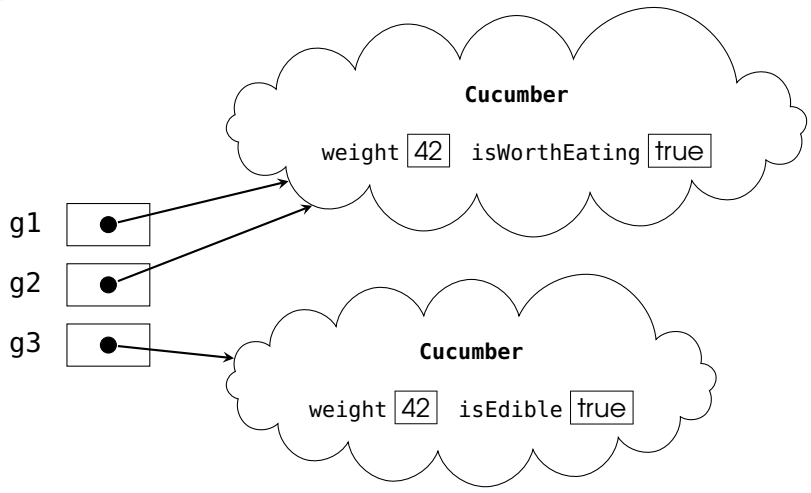
**Solution Task 12.** *Method* `equals`.

a)

1. A class `Cucumber` is created with parameters `weight` of type `Int` and `isEatable` of type `Boolean`.
2. `g1` is assigned an instance of `Cucumber` with `weight = 42` and `isEatable = true`.
3. `g2` is assigned the same `Cucumber` object as `g1`.
4. `g3` is assigned a new `Cucumber` instance with the same parameter values as `g1`.
5. The == (`equals`) method compares `g1` and `g2` and returns **true**.
6. == (`equals`) method compares `g1` and `g3` and returns **false**.
7. `def equals(x\$1: Any): Boolean`

As seen the parameter of `equals` is of type `Any`. Since the program does not know the class, it uses `Any.equals`. Unlike primitive types that compare content with `equals`, classes default to reference equality unless otherwise specified. `g1` and `g2` refer to the same object but `g3` points to a different one, so `g1 == g3` is false.

b)




c) -

d) In the first three lines the same actions happen as in subtask $a$. Now when comparing `Cucumber` objects, the `equals` method is overridden by the one defined specifically for `Cucumber`. Since both objects compared are of type `Cucumber`, the case `that: Cucumber` matches. It then compares the weights and edible booleans returning `Boolean` indicating equality, which in both cases is true.

e) In subtask a, `g1 == g3` was false despite content equality. After overriding, this expression returns **`true`**, demonstrating that == now uses content equality.

**Solution Task 13.** *Polynomial.*

a) TODO!!!

b) TODO!!!

**Solution Task 14.** *`Option` as a collection.*

Examples of methods that exist both for `Vector` and `Option`: `foreach`, `filter`, `fold` etc.

The method `contains` returns a `Boolean` indicating whether it holds a value or not.

**Solution Task 15.** *Catch exceptions with **`catch`** in Java and Scala.* TODO!!!

**Solution Task 16.** *Polynomial, continuation: reduction.*

**Solution Task 17.** *Type-safe content test with method ===.*

**Solution Task 18.** *Override `equals` with content equality also for non-final classes.*

**Solution Task 19.** *Override equals with inheritance.*

**Solution Task 20.** *Special matchings.* TODO!!!

**Solution Task 21.** *Extractors.* TODO!!!

**Solution Task 22.** *Polynomial, continuation: polynomial division.* TODO!!!

# 7. Solution `Excercise07`

## 7.1 Basic Tasks; preparation for the lab

**Solution Task 1.** *Match concepts with description.*

| | | | | |
|---|---|---|---|---|
| element | 1 | $\rightsquigarrow$ | G | object in a data structure |
| collection | 2 | $\rightsquigarrow$ | B | data structure with elements of same type |
| collection library | 3 | $\rightsquigarrow$ | J | many ready-made collections with different properties |
| sequence(collection) | 4 | $\rightsquigarrow$ | L | zero or more elements of same type in a specific order |
| sequence algorithm | 5 | $\rightsquigarrow$ | I | solution to problem using a sequence collection |
| order | 6 | $\rightsquigarrow$ | A | defines how elements of a type are arranged |
| sorting | 7 | $\rightsquigarrow$ | C | algorithm arranging elements in a specific order |
| searching | 8 | $\rightsquigarrow$ | D | algorithm finding elements according to a search criterion |
| linear search | 9 | $\rightsquigarrow$ | F | searching a sequence until element found |
| counting | 10 | $\rightsquigarrow$ | H | algorithm counting elements with certain properties |
| time complexity | 11 | $\rightsquigarrow$ | E | how execution time grows with problem size |
| memory complexity | 12 | $\rightsquigarrow$ | K | how memory usage grows with problem size |

**Solution Task 2.** *Different sequence collections.*

| | | | | |
|---|---|---|---|---|
| `Vector` | 1 | $\rightsquigarrow$ | B | immutable, quickly returns any modified collection |
| `List` | 2 | $\rightsquigarrow$ | C | immutable, quickly returns new collection modified at start |
| `Array` | 3 | $\rightsquigarrow$ | D | primitive, mutable, fast indexing, fixed size |
| `ArrayBuffer` | 4 | $\rightsquigarrow$ | A | mutable, fast indexing, can change size |
| `ListBuffer` | 5 | $\rightsquigarrow$ | E | mutable, fast to modify at start |

**Solution Task 3.** *Using sequence collections.*

a)

| | | | | |
|---|---|---|---|---|
| `x +: xs` | 1 | $\rightsquigarrow$ | G | `Vector(0, 1, 2, 3)` |
| `xs +: x` | 2 | $\rightsquigarrow$ | L | `error: value +: is not a member of Int` |
| `xs :+ x` | 3 | $\rightsquigarrow$ | J | `Vector(1, 2, 3, 0)` |
| `xs ++ xs` | 4 | $\rightsquigarrow$ | M | `Vector(1, 2, 3, 1, 2, 3)` |
| `xs.indices` | 5 | $\rightsquigarrow$ | E | `(0 until 3)` |
| `xs apply 0` | 6 | $\rightsquigarrow$ | C | `1` |
| `xs(3)` | 7 | $\rightsquigarrow$ | I | `java.lang.IndexOutOfBoundsException` |
| `xs.length` | 8 | $\rightsquigarrow$ | O | `3` |
| `xs.take(4)` | 9 | $\rightsquigarrow$ | F | `Vector(1, 2, 3)` |
| `xs.drop(2)` | 10 | $\rightsquigarrow$ | K | `Vector(3)` |
| `xs.updated(0, 2)` | 11 | $\rightsquigarrow$ | B | `Vector(2, 2, 3)` |
| `xs.tail.head` | 12 | $\rightsquigarrow$ | N | `2` |
| `xs.head.tail` | 13 | $\rightsquigarrow$ | D | `error: value tail is not a member of Int` |
| `xs.isEmpty` | 14 | $\rightsquigarrow$ | H | **`false`** |
| `xs.nonEmpty` | 15 | $\rightsquigarrow$ | A | **`true`** |

b)

| *error* | *type* | *explanation* |
|---|---|---|
| `value +: is not a member of Int` | compile-time error | Operators ending with colon are right-associative. The method call `xs +: x` corresponds in dot notation to `x.+:(xs)` and there is no method named `+:` on integers. |
| `IndexOutOfBoundsException` | runtime error | There are only 3 elements and indexing starts at 0 in sequence collections. |
| `value tail is not a member of Int` | compile-time error | The method `head` returns the first element and integers do not have the sequence collection method `tail`. |

**Solution Task 4.** *Copying sequences.*

a)

| | |
|---|---|
| `xs(0)` | `rs$line5$Mutant@66d766b9` new instances get new hex codes |
| `ys(0).int` | `0` since `ys` contains the same instance as `xs` |
| `zs(0).int` | `5` since `!(xs(0) eq zs(0))` |
| `xs(0) eq ys(0)` | **`true`** because same instance |
| `xs(0) eq zs(0)` | **`false`** since different instances |
| `(ys.toBuffer :+ new Mutant).apply(0).int` | `0` since the non-deep-copied ArrayBuffer copy refers to the same instance at the first position as both `ys` and `xs` and `x(0).int` became zero in an assignment on line 5 in REPL run |

Note that copying with `toArray`, `toVector`, `toBuffer`, etc. *is not deep*, i.e., only instance references are copied, not the instances themselves.

b)

```
def deepCopy(xs: Array[Mutant]): Array[Mutant] =
  val result = Array.ofDim[Mutant](xs.length) // filled with null references
  var i = 0
  while i < xs.length do
    result(i) = new Mutant(xs(i).int) // copy with same content at the same place
    i += 1
  result
```

It is also fine to create the result array with **`new`** `Array[Mutant](xs.length)`. You can also use `size` instead of `length`.

c)

```
scala> class Mutant(var int: Int = 0)
// defined class Mutant

scala> def deepCopy(xs: Array[Mutant]): Array[Mutant] =
     |   val result = Array.ofDim[Mutant](xs.length)
     |   var i = 0
     |   while i < xs.length do
     |     result(i) = new Mutant(xs(i).int)
     |     i += 1
     |   result

scala> val xs = Array.fill(3)(new Mutant)
xs: Array[Mutant] = Array(rs$line$2$Mutant@46a123e4, rs$line$2$Mutant@44bc2449,
rs$line2$Mutant@3c28e5b6)

scala> val ys = deepCopy(xs)
ys: Array[Mutant] = Array(rs$line$2$Mutant@14b8a751, rs$line2$Mutant@7345f97d,
rs$line$2$Mutant@554566a8)

scala> xs(0).int = 5

scala> ys(0).int
val res0: Int = 0
```

d) No, since elements cannot be mutated one can safely share references between collections. There is no possibility that changes affect multiple collections at the same time. However, deep copies of collections with mutable elements are commonly (though often time-consuming) made to ensure the original collection does not change.

**Solution Task 5.** *Updating sequences.*

a)

| | | | |
|---|---|---|---|
| `{ buf(0) = -1; buf(0) }` | 1 | $\rightsquigarrow$ D | `-1` |
| `{ xs(0) = -1; xs(0) }` | 2 | $\rightsquigarrow$ A | `error: value update is not a member` |
| `buf.update(1, 5)` | 3 | $\rightsquigarrow$ F | `(): Unit` |
| `xs.updated(0, 5)` | 4 | $\rightsquigarrow$ B | `Vector(5, 2, 3, 4)` |
| `{ buf += 5; buf }` | 5 | $\rightsquigarrow$ C | `ArrayBuffer(-1, 5, 3, 4, 5)` |
| `{ xs += 5; xs }` | 6 | $\rightsquigarrow$ G | `error: value += is not a member` |
| `xs.patch(1,Vector(-1,5),3)` | 7 | $\rightsquigarrow$ E | `Vector(1, -1, 5)` |
| `xs` | 8 | $\rightsquigarrow$ H | `Vector(1, 2, 3, 4)` |

b)

```
def insert(xs: Array[Int], elem: Int, pos: Int): Array[Int] =
  xs.patch(from = pos, other = Array(elem), replaced = 0)
```

c) Pseudocode below is written so that it compiles though it is incomplete.

```
def insert(xs: Array[Int], elem: Int, pos: Int): Array[Int] =
  val result = ??? /* new array with space for one more element than xs */
  var i = 0
  while(???){/* copy elements before pos and increment i */}
  if i < result.length then /* put elem in result at position i */
  while(???){/* copy the rest */}
  result
```

d)

```
def insert(xs: Array[Int], elem: Int, pos: Int): Array[Int] =
  val result = new Array[Int](xs.length + 1)
  var i = 0
  while i < pos && i < xs.length do  { result(i) = xs(i); i += 1}
  if i < result.length then { result(i) = elem; i += 1 }
  while i < result.length do { result(i) = xs(i - 1); i += 1}
  result
```

```
scala> insert(Array(1, 2), 0, pos = -1)
val res2: Array[Int] = Array(0, 1, 2)

scala> insert(Array(1, 2), 0, pos = 0)
val res3: Array[Int] = Array(0, 1, 2)

scala> insert(Array(1, 2), 0, pos = 1)
```

```
val res4: Array[Int] = Array(1, 0, 2)

scala> insert(Array(1, 2), 0, pos = 2)
val res5: Array[Int] = Array(1, 2, 0)

scala> insert(Array(1, 2), 0, pos = 42)
val res7: Array[Int] = Array(1, 2, 0)
```

**Solution Task 6.** *Comparing strings in Scala.*

a)

```
true
true
true
true
true
false
```

b) *s1* comes first.

**Solution Task 7.** *Linear search according to different criteria.*

a)

| | | | |
|---|---|---|---|
| `xs.indexOf(0)` | 1 | $\rightsquigarrow$ F | `5` |
| `xs.indexOf(6)` | 2 | $\rightsquigarrow$ B | `-1` |
| `xs.indexWhere(_ < 2)` | 3 | $\rightsquigarrow$ H | `4` |
| `xs.indexWhere(_ != 5)` | 4 | $\rightsquigarrow$ I | `1` |
| `xs.find(_ == 1)` | 5 | $\rightsquigarrow$ D | `Some(1)` |
| `xs.find(_ == 6)` | 6 | $\rightsquigarrow$ J | `None` |
| `xs.contains(0)` | 7 | $\rightsquigarrow$ C | **`true`** |
| `xs.filter(_ == 1)` | 8 | $\rightsquigarrow$ A | `Vector(1, 1)` |
| `xs.filterNot(_ > 1)` | 9 | $\rightsquigarrow$ E | `Vector(1, 0, 1)` |
| `xs.zipWithIndex.filter(_._1 == 1).map(_._2)` | 10 | $\rightsquigarrow$ G | `Vector(4, 6)` |

b) With a boolean variable `found`:

```scala
def indexOf(xs: Vector[String], p: String => Boolean): Int =
  var found = false
  var i = 0
  while i < xs.length && !found do
      found = p(xs(i))
      i += 1
  if found then i - 1 else -1
```

Or without `found`:

```scala
def indexOf(xs: Vector[String], p: String => Boolean): Int =
  var i = 0
  while i < xs.length && !p(xs(i)) do i += 1
```

```
  if i == xs.length then -1 else i
```

Or maybe you want to start searching from the end; the solution below is probably easier to understand (?) and definitely more concise, but it does *not* fulfill the requirement to return the index of the *first* occurrence as stated in the task. However, if the context allows returning *any* index for which the predicate holds, or if you want to search backwards, this is fine:

```
def indexOf(xs: Vector[String], p: String => Boolean): Int =
  var i = xs.length - 1
  while i >= 0 && !p(xs(i)) do i -= 1
  i
```

There are many other ways to solve this. When implementing algorithms, both in programming exams and in professional development, there are often many different solutions with various properties, advantages, and disadvantages. The important thing is that the solution works well according to requirements, the code is understandable, and the implementation is not so inefficient that users get tired waiting for the result...

**Solution Task 8.** *Lab preparation: Implement integer counting in Array.*

a)

```
def registerDiceRoll(xs: Seq[Int]): Vector[Int] =
  val result = Array.fill(6)(0)
  xs.foreach{ x =>
    require(x >= 1 && x <= 6, "dice roll must be between 1 & 6")
    result(x - 1) += 1
  }
  result.toVector
```

b)

```
scala> registerDiceRoll(toss(1000))
val res0: Vector[Int] = Vector(171, 163, 166, 152, 184, 164)

scala> registerDiceRoll(toss(1000))
val res1: Vector[Int] = Vector(163, 161, 158, 174, 161, 183)
```

**Solution Task 9.** *Built-in methods for sorting.*

| | | | | |
|---|---|---|---|---|
| `'a' < 'A'` | 1 | ⇝ | E | `false` |
| `"AÄÖö" < "AÅÖö"` | 2 | ⇝ | H | `true` |
| `xs.sorted.head` | 3 | ⇝ | C | `-1` |
| `xs.sorted.reverse.head` | 4 | ⇝ | G | `3` |
| `ys.sorted.head` | 5 | ⇝ | I | `"ak"` |
| `zs.indexOf('a')` | 6 | ⇝ | B | `1` |
| `ps.sorted.head.firstName.take(2)` | 7 | ⇝ | D | `error: ...` |
| `ps.sortBy(_.firstName).apply(1).firstName.take(2)` | 8 | ⇝ | A | `"ka"` |
| `xs.sortWith((x1,x2) => x1 > x2).indexOf(3)` | 9 | ⇝ | F | `0` |

It will fail in the expression trying to sort a sequence of `Person` instances directly with `sorted`:

```
scala> ps.sorted
No implicit Ordering defined for Person.
```

It fails because the compiler cannot find any ordering definition for your own classes. Later in the course we will see how to create custom orderings if you want `sorted` to work on sequences of your own classes, but often `sortBy` and `sortWith` suffice.

**Solution Task 10.** *Built-in shuffle method.*

a) `Random.shuffle` returns a new shuffled sequence collection of the same type. The original collection's order is not affected.

b) Example usage of `random.shuffle`:

```
scala> import scala.util.Random

scala> val xs = Vector("Sten", "Sax", "Påse")
val xs: Vector[String] = Vector(Sten, Sax, Påse)

scala> (1 to 10).foreach(_ => println(Random.shuffle(xs).mkString(" ")))
Sax Påse Sten
Sten Påse Sax
Sten Sax Påse
Sten Sax Påse
Sten Påse Sax
Sten Påse Sax
Sax Sten Påse
Sten Påse Sax
Sax Påse Sten
Sax Påse Sten

scala> (1 to 5).map(_ => Random.shuffle(1 to 6))
val res1: IndexedSeq[IndexedSeq[Int]] =
  Vector(Vector(5, 2, 1, 4, 3, 6), Vector(6, 5, 4, 2, 1, 3),
  Vector(3, 1, 4, 6, 5, 2), Vector(3, 2, 6, 5, 1, 4),
  Vector(5, 3, 4, 6, 1, 2))

scala> (1 to 1000).map(_ => Random.shuffle(1 to 6).head).count(_ == 6)
val res2: Int = 168
```

**Solution Task 11.** *Repeated parameters.*

a)

```
scala> def stringSizes(xs: String*): Vector[Int] = xs.map(_.size).toVector
def stringSizes(xs: String*): Vector[Int]

scala> stringSizes("hej")
val res0: Vector[Int] = Vector(3)

scala> stringSizes("hej", "on", "dough", "")
val res1: Vector[Int] = Vector(3, 2, 3, 0)

scala> stringSizes()
val res2: Vector[Int] = Vector()
```

Calling with an empty argument list gives an empty integer sequence.

b)

```
scala> val xs = Vector("hej","on","dough", "")
val xs: Vector[String] = Vector(hej, on, dough, "")

scala> stringSizes(xs: _*)
val res0: Vector[Int] = Vector(3, 2, 5, 0)

scala> stringSizes(Vector(): _*)
val res1: Vector[Int] = Vector()
```

Yes, it works fine with an empty sequence.

## 7.2 Extra Tasks; additional practice

**Solution Task 12.** *Counting boolean values. Flipping a coin.*

a)

```
def registerCoinFlips(xs: Seq[Boolean]): (Int, Int) =
  val result = Array.fill(2)(0)
  xs.foreach(x => if (x) result(0) += 1 else result(1) += 1)
  (result(0), result(1))
```

b)

**Solution Task 13.** *Copying and appending at the end.*

```
def copyAppend(xs: Array[Int], x: Int): Array[Int] =
  val ys = new Array[Int](xs.length + 1)
  var i = 0
  while i < xs.length do
    ys(i) = xs(i)
    i += 1
  ys(xs.length) = x
  ys
```

The two bugs in the algorithm are (1) the condition should be strictly less than and (2) the loop variable increment is missing inside the loop.

**Solution Task 14.**

```
def seqReverseCopy(xs: Array[Int]): Array[Int] =
  val n = xs.length
  val ys = new Array[Int](n)
  var i = 0
  while i < n do
    ys(n - i - 1) = xs(i)
    i += 1
  ys
```

a)

```
def seqReverseCopy(xs: Array[Int]): Array[Int] =
  val n = xs.length
  val ys = new Array[Int](n)
  for i <- (n - 1) to 0 by -1 do
    ys(n - i - 1) = xs(i)
  ys
```

**Solution Task 15.** *Copy all except one.*

```
def removeCopy(xs: Array[Int], pos: Int): Array[Int] =
  val n = xs.size
  val ys = Array.fill(n - 1)(0)
```

**Input** : A sequence $xs$ of type `Array[Int]` and $pos$
**Output**: A new sequence of type `Array[Int]` which is a copy of $xs$ but with the element at position $pos$ removed

1 $n \leftarrow$ number of elements $xs$
2 $ys \leftarrow$ a new `Array[Int]` with space for $n-1$ elements
3 **for** $i \leftarrow 0$ **to** $pos-1$ **do**
4 | $ys(i) \leftarrow xs(i)$
5 **end**
6 $ys(pos) \leftarrow x$
7 **for** $i \leftarrow pos+1$ **to** $n-1$ **do**
8 | $ys(i-1) \leftarrow xs(i)$
9 **end**
10 $ys$

```
  for i <- 0 until pos do
    ys(i) = xs(i)
  for i <- (pos + 1) until n do
    ys(i - 1) = xs(i)
  ys
```

**Solution Task 16.** *In-place removal in array.*

**Input** : A sequence $xs$ of type `Array[Int]`, a position $pos$, and a padding value $pad$
**Output**: An updated sequence of $xs$ where the element at position $pos$ is removed and subsequent elements moved one step towards lower index with the last element assigned the value $pad$

1 $n \leftarrow$ number of elements in $xs$
2 **for** $i \leftarrow pos+1$ **to** $n-1$ **do**
3 | $xs(i-1) \leftarrow xs(i)$
4 **end**
5 $xs(n-1) \leftarrow pad$

```
def remove(xs: Array[Int], pos: Int, pad: Int = 0): Unit =
  val n = xs.size
  for i <- (pos + 1) until n do
    xs(i - 1) = xs(i)
  xs(n - 1) = pad
```

**Solution Task 17.** *Copying and inserting.*

a)

```
def insertCopy(xs: Array[Int], x: Int, pos: Int): Array[Int] =
  val n = xs.size
  val ys = Array.ofDim[Int](n + 1)
  for i <- 0 until pos do
    ys(i) = xs(i)
```

```
  ys(pos) = x
  for i <- pos until n do
    ys(i + 1) = xs(i)
  ys
```

b) `pos` must be `0`.

c)

```
java.lang.ArrayIndexOutOfBoundsException: -1
```

d) Element `x` is added to the end of the array, i.e., the returned array is larger than the input array.

e)

```
java.lang.ArrayIndexOutOfBoundsException: 5
```

You get `ArrayIndexOutOfBoundsException` since indexing is out of size bounds.

**Solution Task 18.** *In-place insertion in array.*

**Input** : A sequence $xs$ of type `Array[Int]` and integers $x$ and $pos$
**Output** : $xs$ updated in-place, where element $x$ is inserted at $pos$ and subsequent elements are shifted one step, the last element lost

$n \leftarrow$ number of elements in $xs$
$ys \leftarrow$ a clone of $xs$
$xs(pos) \leftarrow x$
**for** $i \leftarrow pos + 1$ **to** $n - 1$ **do**
  $xs(i) \leftarrow ys(i - 1)$
**end**

```
def insertDropLast(xs: Array[Int], x: Int, pos: Int): Unit =
  val n = xs.size
  val ys = xs.clone
  xs(pos) = x
  for i <- pos + 1 until n do
    xs(i) = ys(i - 1)
```

**Solution Task 19.** *More built-in methods for linear search.*

a)

- `lastIndexOf` is good if you want to search backwards; without it you would otherwise have to use `xs.reverse.indexOf(e)`
- `indexOfSlice(ys)` searches for the index where a whole sequence `ys` begins, unlike `indexOf(e)` which only searches for a single element.
- `segmentLength(p, i)` returns the length of the longest consecutive sequence where all elements satisfy predicate `p`, starting from position `i`
- `xs.maxBy(f)` applies function `f` on all elements in `xs` and returns the element producing the maximum value; similarly `minBy(f)` gives the minimum according to `f`

b) –

## 7.3 Advanced Tasks; challenges

**Solution Task 20.** *Fix Swedish sorting order of ÄÅÖ.* TODO!!!

**Solution Task 21.** *Fibonacci sequence with ListBuffer.*

a)

```
def fib(max: Long): List[Long] =
  val xs = scala.collection.mutable.ListBuffer.empty[Long]
  xs.prependAll(Vector(1, 1))
  while xs.head < max do xs.prepend(xs.take(2).sum)
  xs.reverse.drop(1).toList
```

b)

```
scala> fib(Int.MaxValue).size
val res0: Int = 46
```

c)

```
def fibBig(max: BigInt): List[BigInt] =
  val xs = scala.collection.mutable.ListBuffer.empty[BigInt]
  xs.prependAll(Vector(BigInt(1), BigInt(1)))
  while xs.head < max do xs.prepend(xs.take(2).sum)
  xs.reverse.drop(1).toList
```

```
scala> fibBig(Long.MaxValue).size
val res0: Int = 92

scala> fibBig(BigInt(Long.MaxValue).pow(64)).size
val res1: Int = 5809

scala> fibBig(BigInt(Long.MaxValue).pow(128)).last
val res2: BigInt = 46657280552835544919455361110286315395072000518604554717752

scala> fibBig(BigInt(Long.MaxValue).pow(128)).last.toString.size
val res3: Int = 2428

scala> fibBig(BigInt(Long.MaxValue).pow(256)).last.toString.size
val res4: Int = 4856

scala> fibBig(BigInt(Long.MaxValue).pow(1024)).last.toString.size
java.lang.OutOfMemoryError: Java heap space
```

**Solution Task 22.** *In-place reverse sequence.*

```
def reverseChars(xs: Array[Char]): Unit =
  val n = xs.length
  for i <- 0 to (n/2 - 1) do
    val temp = xs(i)
    xs(i) = xs(n - i - 1)
    xs(n - i - 1) = temp
```

**Solution Task 23.** *Palindrome predicate.*

a) Reversing with `reverse` can require one traversal plus memory for the copy. The content comparison requires another traversal. (This is not a big problem since the world's longest palindrome is no longer than 19 letters and is an obscure Finnish word rarely uttered in daily speech. Which one?)

b)

```
def isPalindrome(s: String): Boolean =
  val n = s.length
  var foundDiff = false
  var i = 0
  while i < n/2 && !foundDiff do
    foundDiff = s(i) != s(n - i - 1)
    i += 1
  !foundDiff
```

**Solution Task 24.** *More useful sequence collection methods.*

```
scala> val xs = Vector.tabulate(10)(i => math.pow(2, i).toInt)
xs: Vector[Int] = Vector(1, 2, 4, 8, 16, 32, 64, 128, 256, 512)

scala> xs.forall(_ < 1024)
val res0: Boolean = true

scala> xs.exists(_ == 3)
val res1: Boolean = false

scala> xs.count(_ > 64)
val res2: Int = 3

scala> xs.zipWithIndex.take(5)
val res3: Vector[(Int, Int)] = Vector((1,0), (2,1), (4,2), (8,3), (16,4))
```

**Solution Task 25.** *Arrays don't behave, but `ArraySeqs` do!*

a) `xs` provides content equality and is of type `Seq[Int]` with underlying type `ArraySeq[Int]`. You cannot assign elements in an `ArraySeq`, as it is immutable. It does not update when the original array changes.

```
scala> val as1 = Array(1,2,3)
val as1: Array[Int] = Array(1, 2, 3)

scala> val as2 = Array(1,2,3)
val as2: Array[Int] = Array(1, 2, 3)


scala> val (xs1, xs2) = (as1.toSeq, as2.toSeq)
val xs1: Seq[Int] = ArraySeq(1, 2, 3)
val xs2: Seq[Int] = ArraySeq(1, 2, 3)

scala> as1 == as2
val res0: Boolean = false

scala> xs1 == xs2
```

```
val res1: Boolean = true

scala> as1(0) = 42

scala> xs1
val res2: Seq[Int] = ArraySeq(1, 2, 3)

scala> xs1(0) = 42
value update is not a member of Seq[Int]
```

b) With repeated parameters you get an `ArraySeq`.

```
scala> def f(xs: Int*) = xs
def f(xs: Int*): Seq[Int]

scala> println(f(1,2,3))
ArraySeq(1, 2, 3)
```

c) It is not possible to have a generic array as function result without attaching a context bound `ClassTag` to the type parameter so that the compiler can generate code for type conversion required at JVM runtime. See example here:
http://docs.scala-lang.org/overviews/collections/arrays.html

**Solution Task 26.** The sequence collection `List` is almost twice as fast when processed at the beginning but about 1000 times slower when processed at the end of a sequence with 100000 elements.

Different executions run at different speeds on the JVM, among other things due to optimizations that occur when the JVM warms up and the so-called Just-In-Time compilation does its mighty job. It sometimes suddenly runs significantly slower when the garbage collector is forced to do a time-consuming cleanup of the memory.

**Solution Task 27.** –

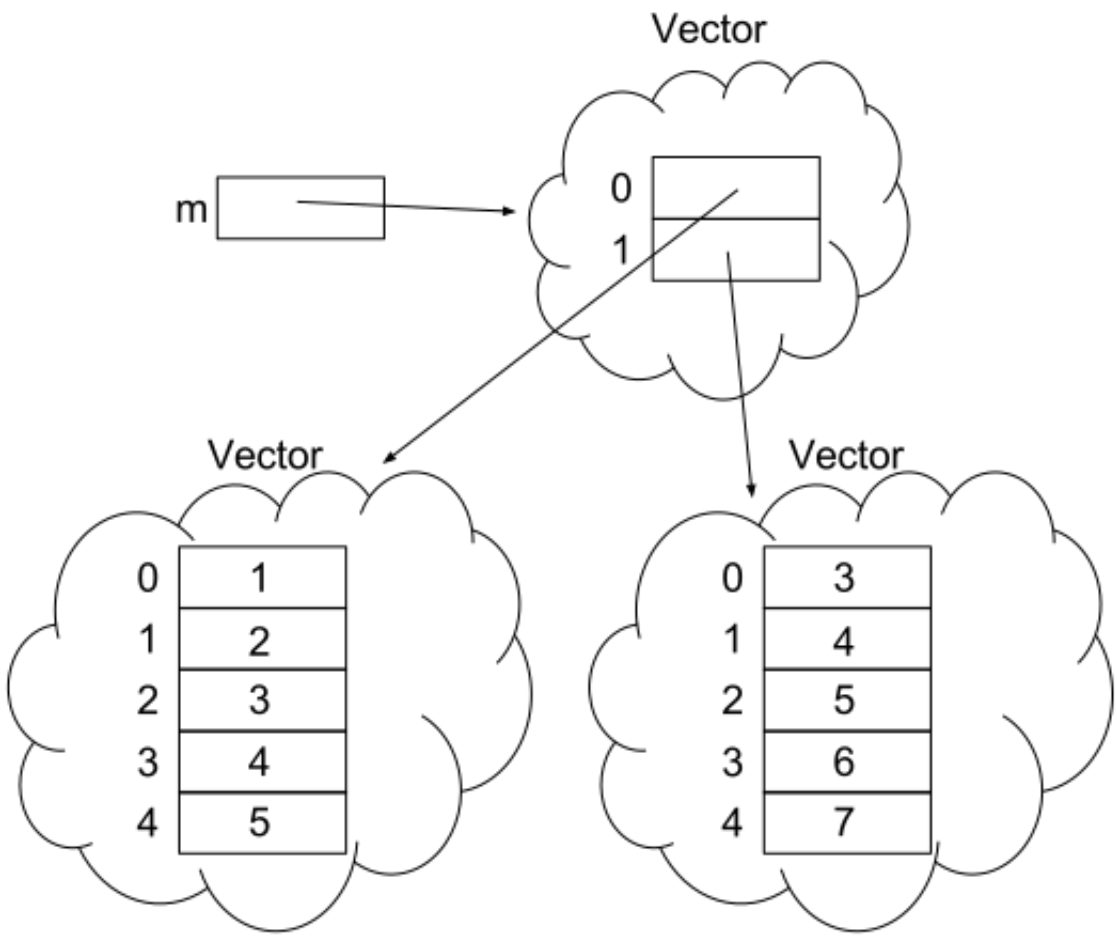
# 8. Solution `Excercise08`

## 8.1 Basic Tasks; preparation for the lab

**Solution Task 1.** *Match concepts with descriptions.*

| | | | | |
|---|---|---|---|---|
| matrix | 1 | $\rightsquigarrow$ | A | indexable 2D data structure |
| row vector | 2 | $\rightsquigarrow$ | F | matrix of dimension $1 \times m$ with $m$ horizontal values |
| column vector | 3 | $\rightsquigarrow$ | G | matrix of dimension $m \times 1$ with $m$ vertical values |
| column | 4 | $\rightsquigarrow$ | C | another word for column |
| generic | 5 | $\rightsquigarrow$ | B | has abstract type parameter, type is general |
| type argument | 6 | $\rightsquigarrow$ | D | concrete type, binds to type parameter at compile time |
| type inference | 7 | $\rightsquigarrow$ | E | compiler deduces type from context |

**Solution Task 2.** *Create matrices using nested collections.*




a)

Type: `Vector[Vector[Int]]`

Value: `Vector(Vector(1, 2, 3, 4, 5), Vector(3, 4, 5, 6, 7))`

Dimensions: $2 \times 5$

In mathematics, indexing is conventionally with 1 as the lowest index. In Scala, the lowest index is 0, called 0-based indexing. This is not the case in all programming languages; you can read more at https://en.wikipedia.org/wiki/Array_data_type#Index_origin.

b)

```
scala> val m = Vector((1 to 5).toVector, (3 to 7).toVector)
m: Vector[Vector[Int]] = Vector(Vector(1, 2, 3, 4, 5), Vector(3, 4, 5, 6, 7))

scala> m.apply(0).apply(1)
res4: Int = 2

scala> m(1)
res5: Vector[Int] = Vector(3, 4, 5, 6, 7)

scala> m(1)(4)
res6: Int = 7
```

c)
m2: `Vector[Vector[Int]]`
m3: `Vector[Vector[Int | Double]]`
m4: `Vector[Vector[Int | Double | String]]`
m5: `Vector[Vector[Int]]`

d) m5, $42 \times 2$

**Solution Task 3.** *Create and iterate over matrices.*

a)

```
def throwDie: Int = (math.random() * 6).toInt + 1
```

Or:

```
def throwDie: Int = scala.util.Random.nextInt(6) + 1
```

b) Matrix dimension in mathematical notation: $1000 \times 5$, which corresponds to a matrix with 1000 rows and 5 columns.

c)

```
ds1: IndexedSeq[IndexedSeq[Int]]
ds2: IndexedSeq[IndexedSeq[Int]]
ds3: IndexedSeq[Vector[Int]]
ds4: IndexedSeq[Vector[Int]]
ds5: Vector[Vector[Int]]
ds6: Vector[Vector[Int]]
```

Both `IndexedSeq` and `Vector` above are in the package `scala.collection.immutable`.

d)

```
def roll(n: Int) = Vector.fill(n)(throwDie).sorted
```

e)

```
def isYatzy(xs: Vector[Int]): Boolean = xs.forall(_ == xs(0))
```

f)

```
def diceMatrix(m: Int, n: Int): Vector[Vector[Int]] =
  Vector.fill(m)(roll(n))
```

g)

```
def diceMatrixToString(xss: Vector[Vector[Int]]): String =
  xss.map(_.mkString(" ")).mkString("\n")
```

h)

```
def filterYatzy(xss: Vector[Vector[Int]]): Vector[Vector[Int]] =
  xss.filter(isYatzy)
```

i)

```
def yatzyPips(xss: Vector[Vector[Int]]): Vector[Int] =
  filterYatzy(xss).map(_.head)
```

**Solution Task 4.** *An immutable, generic matrix class for this week's lab* `life`.

a) The type of `m` becomes `Matrix`.

b) The type of `e` becomes `String`.

c) You need to change 3 places from `String` to `Int`.

d) Generic matrix `Matrix[T]` for elements of any type `T`:

```
case class Matrix[T](data: Vector[Vector[T]]):
  def apply(row: Int, col: Int): T = data(row)(col)

object Matrix:
  def fill[T](dim: (Int, Int))(value: T): Matrix[T] =
    Matrix[T](Vector.fill(dim._1, dim._2)(value))
```

e) Thanks to compiler type inference, `bm` has type `Matrix[Boolean]`.

f) The type of `be` becomes `Boolean`.

g) h) i) j) k) are all implemented in the code below:

```
case class Matrix[T](data: Vector[Vector[T]]):
  require(data.forall(row => row.length == data(0).length))

  val dim: (Int, Int) = (data.length, data(0).length)

  def apply(row: Int, col: Int): T = data(row)(col)

  def updated(row: Int, col: Int)(value: T): Matrix[T] =
    Matrix(data.updated(row, data(row).updated(col, value)))

  def foreachIndex(f: (Int, Int) => Unit): Unit =
    for r <- data.indices; c <- data(r).indices do f(r, c)
```

```
  override def toString =
    s"""Matrix of dim $dim:\n${ data.map(_.mkString(" ")).mkString("\n") }"""

object Matrix:
  def fill[T](dim: (Int, Int))(value: T): Matrix[T] =
    Matrix[T](Vector.fill(dim._1, dim._2)(value))
```

## 8.2 Extra Tasks; additional practice

**Solution Task 5.** *Imperative matrix algorithms.*

a)

```
def isYatzy(xs: Vector[Int]): Boolean =
  var foundDiff = false
  var i = 0
  while (i < xs.size && !foundDiff) do
    foundDiff = xs(i) != xs(0)
    i += 1
  end while
  !foundDiff
```

b) The function goes through each matrix row, and for each it iterates over each element in the row and appends to the `StringBuilder` object. If the element is not the last in its row, a space is appended; otherwise, a newline is appended. Exception is the last row, where no newline is appended. Finally, the `StringBuilder` is converted to a `String` and returned.

If `xss` is empty, `xss.indices` is an empty `Range` and the outer for-loop is skipped, returning an empty string. If all rows are empty, the inner for-loops are skipped, with the same result.

*Advantage:* `StringBuilder` is faster for appending to large strings (though not noticeable here because the string is small).

*Disadvantage:* The `StringBuilder` code is often perceived as harder to read.

c)

```
def filterYatzy(xss: Vector[Vector[Int]]): Vector[Vector[Int]] =
  var result: Vector[Vector[Int]] = Vector()
  for i <- xss.indices if isYatzy(xss(i)) do result = result :+ xss(i)
  result
```

d) Each loop iteration yields a vector `xss(i)` if the filter condition is satisfied, resulting in a vector of vectors that are yatzy throws.

**Solution Task 6.** *String table with column headers.*

a)

```
case class Table(
  data: Vector[Vector[String]],
  headings: Vector[String],
  sep: Char
):

  val dim: (Int, Int) = (data.size, headings.size)

  def apply(r: Int, c: Int): String = data(r)(c)

  def row(r: Int): Vector[String]= data(r)

  def col(c: Int): Vector[String] = data.map(r => r(c))
```

```
  lazy val indexOfHeading: Map[String, Int] = headings.zipWithIndex.toMap

  def col(h: String): Vector[String] = col(indexOfHeading(h))

  def values(h: String): Vector[String] = col(h).distinct.sorted

  override def toString: String =
    val s = sep.toString
    headings.mkString(s) + "\n" +data.map(_.mkString(s)).mkString("\n")

object Table:
  def fromFile(fileName: String, sep: Char = ';'): Table =
    val lines = scala.io.Source.fromFile(fileName).getLines.toVector
    val matrix= lines.map(_.split(sep).toVector)
    new Table(matrix.tail, matrix.head, sep)
```

b)

```
@main
def run(fileName: String, separator: String): Unit =
  require(separator.length == 1, "separator ska vara exakt ett tecken")
  val t = Table.fromFile(fileName, separator.head)
  val counts: Vector[Vector[String]] =
    (0 until t.dim._2)
      .map(i => t.values(t.headings(i))
      .map(x => s"$x: ${t.col(i).count(_ == x)}"))
      .toVector
  for (i <- 0 until t.dim._2) do
    println(s"\nColumn: ${i + 1}, ${t.headings(i)}:")
    for (j <- 0 until counts(i).length) do
      println(counts(i)(j))
```

**Solution Task 7.** *Create a yatzy game for terminal use.*

–

## 8.3 Advanced Tasks; challenges

**Solution Task 8.** *Generic functions.*

a)

1. –
2. The string representation of `42` is reversed
3. `"hej"` is reversed — `toString` of a string returns the string itself
4. –
5. The string representation of the `Cucumber` object is reversed
6. The function's type parameter does not match the parameter's type: `42` is not a string
7. Implicit conversion to `Double` happens to match the type parameter, giving a string representation with decimal

b)

1. A function is defined that takes two other functions as arguments, composes them, and applies the composed function to a third argument.
2. A function that increments an integer by 1 is defined.
3. A function that halves a floating point number is defined.
4. `42` is passed to `inc()`, and the result (`43`) is passed to `half()`. Inside `half()`, implicit conversion to `Double` happens since the number is divided by a floating point number (2.0), resulting in `43.0 / 2.0`, that is, `21.5`.
5. The result from `half()` is of type `Double`, but `inc()` expects an `Int`. Since floating point numbers cannot be implicitly converted to integers without loss of information, no implicit conversion occurs, and a compilation error results.

c)

```
def inc(x: Double): Double = x + 1.0
```

Now the compilation error moves to line 4 instead, which can be fixed with the following change:

```
def half(x: Double): Double = x / 2.0
```

**Solution Task 9.** *Generic classes.*

a) –

b)

```
class Cell[T](var value: T):
  override def toString = "Cell(" + value + ")"
  def concat[U](that: Cell[U]): Cell[String] =
    Cell(s"$value${that.value}")
```

c) Only cells with the same type parameter can now be concatenated. Because `concat()` returns an object of type `Cell[String]`, an odd number of cells of any other type than `String` cannot be concatenated. If the number is even, they can be concatenated pairwise and then concatenate the resulting `Cell[String]` objects, but that is somewhat cumbersome.

**Solution Task 10.** *Implement more generic methods in `Matrix[T]`.*

```
case class Matrix[T](data: Vector[Vector[T]]):
  require(data.forall(row => row.size == data(0).size))

  val dim: (Int, Int) = (data.length, data(0).length)

  def apply(row: Int, col: Int): T = data(row)(col)

  def updated(row: Int, col: Int)(value: T): Matrix[T] =
    Matrix(data.updated(row, data(row).updated(col, value)))

  def foreach(f: T => Unit): Unit = data.foreach(_.foreach(f))

  def foreachIndex(f: (Int, Int) => Unit): Unit =
    for r <- data.indices; c <- data(r).indices do f(r, c)

  def map[U](f: T => U): Matrix[U] = Matrix(data.map(_.map(f)))

  def mapIndex[U](f: (Int, Int) => U): Matrix[U] =
    var result = Matrix.fill(dim)(f(0,0))
    for
      r <- data.indices
      c <- data(r).indices
    do
      result = result.updated(r, c)(f(r, c))
    end for
    result

  override def toString =
    s"""Matrix of dim $dim:\n${ data.map(_.mkString(" ")).mkString("\n") }"""

object Matrix:
  def fill[T](dim: (Int, Int))(value: T): Matrix[T] =
    Matrix[T](Vector.fill(dim._1, dim._2)(value))
```

# 9. Solution `Excercise09`

## 9.1 Basic Tasks; preparation for the lab

**Solution Task 1.** *Match concepts with description.*

| | | | | |
|---|---|---|---|---|
| set | 1 | ⇝ | H | unordered collection with unique elements |
| key-value table | 2 | ⇝ | F | unordered collection of mappings with unique keys |
| mapping | 3 | ⇝ | G | `key -> value` |
| key | 4 | ⇝ | C | a unique identifier |
| persistence | 5 | ⇝ | D | property of remaining after program ends |
| serialize | 6 | ⇝ | E | encode object into decodable sequence of symbols |
| deserialize | 7 | ⇝ | B | decode symbol sequence and recreate object in memory |
| linear search | 8 | ⇝ | A | search sequence until criterion met |

**Solution Task 2.** *What is a set?* A set is a collection that can quickly answer whether a certain element belongs to the collection or not. The elements in a set are unique. Adding an already existing element is ignored. A set is not a sequence, since traversal with e.g. `map` or `foreach` does not (necessarily) happen in the order in which the elements were given when the set was constructed or updated.

**Solution Task 3.** *Using sets.*

| | | | | |
|---|---|---|---|---|
| `Set(1, 2) ++ Set(1, 2)` | 1 | ⇝ | I | `Set(1, 2)` |
| `(1 to 3).toSet` | 2 | ⇝ | G | `Set(1) + 2 + 3` |
| `Vector.fill(3)(1).toSet` | 3 | ⇝ | F | `Set(1, 2) - 2` |
| `Set(1, 2, 3) diff Set(1, 2)` | 4 | ⇝ | B | `Set(3)` |
| `(1 to 7).toSet.apply(8)` | 5 | ⇝ | H | `false` |
| `Set(1, 2, 3).sorted` | 6 | ⇝ | D | `error: ...` |
| `Set(2,4) subsetOf (1 to 7).toSet` | 7 | ⇝ | E | `true` |
| `Set(1, -1, 2, -2).map(_.abs).sum` | 8 | ⇝ | A | `3` |
| `Set(1, 1, 1, 1, 1, 5).sum` | 9 | ⇝ | C | `6` |

**Solution Task 4.** *Count unique words using a set.*

a)

```
scala> val hej = "hej hej hemskt mycket hej"
scala> val n = hej.split(' ').toSet.size
n: Int = 3
```

b) The method `distinct` returns a sequence with unique elements and preserved original order.

**Solution Task 5.** *Creating 2-tuples with the method `->` which can be read as "maps to".*

a) Yes, the factory method returns a completely normal pair:

```
scala> val hereILive = "Skåne" -> "Lund"
val hereILive: (String, String) = (Skåne,Lund)

scala> hereILive._1
val res0: String = Skåne

scala> hereILive._2
val res1: String = Lund
```

b)

```
val capital = Vector(
  "Sverige"  -> "Stockholm",
  "Danmark"  -> "Köpenhamn",
  "Grönland" -> "Nuuk",
  "Skåne"    -> "Lund"
)
```

c)

```
scala> capital(3)._2
val res2: String = Lund
```

**Solution Task 6.** *Linear search for key in sequence of mappings.*

a)

```
def lookupIndex(xs: Vector[(String, String)])(key: String): Int =
  xs.indexWhere(_._1 == key)
```

b)

```
scala> val i = lookupIndex(capital)("Skåne")
val i: Int = 3

scala> capital(i)._2
val res2: String = Lund
```

Or with functions as reusable partial solutions:

```
scala> val indexOf = lookupIndex(capital) _

scala> def capital(key: String) = capital(indexOf(key))._2

scala> capital("Skåne")
val res3: String = Lund

scala> capital("Sverige")
val res4: String = Stockholm
```

**Solution Task 7.** *Key-value table.*

```
scala> telnr("Fröken Ur")
```

```
val res0: Long = 464690510

scala> :type telnr
Map[String,Long]

scala> :type telnr.toVector
Vector[(String, Long)]
```

**Solution Task 8.** *Using key-value table.*

a) No; the key-value pairs are stored in a special order determined by an internal, clever storage principle based on a so-called hash function[3], to achieve fast lookups of values from keys, which normally does not coincide with the order of the sequence from which they were created.

b)

| | | | |
|---|---|---|---|
| `xs(2) + xs(4)` | 1 | $\rightsquigarrow$ A | `8` |
| `ys(0)` | 2 | $\rightsquigarrow$ C | `(10, 11)` |
| `xs(0)` | 3 | $\rightsquigarrow$ I | `NoSuchElementException` |
| `(xs + (0 -> 1)).apply(0)` | 4 | $\rightsquigarrow$ D | `1` |
| `xs.keySet.apply(2)` | 5 | $\rightsquigarrow$ G | `true` |
| `xs isDefinedAt 0` | 6 | $\rightsquigarrow$ H | `false` |
| `xs.getOrElse(0, 7)` | 7 | $\rightsquigarrow$ B | `7` |
| `xs.maxBy(_._2)` | 8 | $\rightsquigarrow$ E | `(16, 17)` |
| `xs.map(p => p._1 -> -p._2)(8)` | 9 | $\rightsquigarrow$ F | `-9` |

**Solution Task 9.** *Registration in mutable key-value table.*

```
class FreqMapBuilder:
  private val register = scala.collection.mutable.Map.empty[String,Int]
  def toMap: Map[String, Int] = register.toMap
  def add(s: String): Unit =
    register.addOne(s -> (register.getOrElse(s, 0) + 1))

object FreqMapBuilder:
  def apply(xs: String*): FreqMapBuilder =
    val result = new FreqMapBuilder
    xs.foreach(result.add)
    result
```

**Solution Task 10.** *The method `sliding`.*

a)

```
scala> val xs = Vector("five", "cucumbers", "is", "more", "than", "four", "toma
val xs: Vector[String] =
  Vector(five, cucumbers, is, more, thane, four, tomatoes)

```

[3] https://sv.wikipedia.org/wiki/Hashfunktion

```
scala> xs.sliding(2).toVector
val res9: Vector[Vector[String]] =
  Vector(Vector(five, cucumbers), Vector(cucumbers, is), Vector(is, more), Vect

scala> xs.sliding(3).toVector
val res10: Vector[Vector[String]] =
  Vector(Vector(five, cucumbers, is), Vector(cucumbers, is, more), Vector(is, m

scala> xs.sliding(10).toVector
val res11: Vector[Vector[String]] =
  Vector(Vector(five, cucumbers, is, more, than, four, tomatoes))
```

`xs.sliding(n).toVector` creates a sequence that contains sequences of length `n` created by taking each element and its `n - 1` following elements.

b)

```
scala> xs.sliding(2).map(ys => ys(0) -> ys(1)).toMap
val res0: Map[String,String] =
  Map(is -> more,
      than -> four,
      four -> tomatoes,
      cucumbers -> is,
      five -> cucumbers,
      more -> than
  )
```

The table can be used to look up the following word. It works because all words are unique. If there had been multiple identical words with different following words, we would need to create a table with keys that map to a collection registering the following words. This we will do in this week's lab.

**Solution Task 11.** *Reading text from file and web servers.*

a)

```
val populationOf = data.tail.map(v => v(0) -> v(1).toInt).toMap
val sizeOf       = data.tail.map(v => v(0) -> v(2).toInt).toMap
val capitalOf    = data.tail.map(v => v(0) -> v(3)).toMap
```

```
scala> capitalOf("Sverige")
res2: String = Stockholm

scala> populationOf("Sverige")
res3: Int = 9223766

scala> sizeOf("Sverige")
res4: Int = 449964
```

```
scala> val filename = "europa.txt"
scala> val xs = io.Source.fromFile(filename, "UTF-8").getLines.toVector
scala> val data = xs.map(_.split(';').toVector)
scala> data.map(_.map(_.take(15).padTo(15,' ')).mkString(" ")).foreach(println)
```

## 9.2 Extra Tasks; additional practice

**Solution Task 12.** –

## 9.3 Advanced Tasks; challenges

**Solution Task 13.** *Registration with `groupBy`.*

a) The method `groupBy` creates a key-value table where the values in the table are a sequence with the elements grouped in a special way. More precisely:

The result of `xs.groupBy(f: K => V)` for a sequence `xs` of type `Vector[K]` is a table of type `Map[V,Vector[K]]` where each element `e` in `xs` is grouped in the same table value if they are equal according to `f(e)`. Each group gets the table key `f(e)`.

*Clever trick:* If you let the function `f` be the identity function that maps each element onto itself, i.e. `x => x`, then the values are grouped in the same sequence if they are equal.

```
scala> val xs = Vector(1, 1, 2, 2, 4, 4, 4).groupBy(x => x > 2)
val xs: Map[Boolean,Vector[Int]] =
  Map(false -> Vector(1, 1, 2, 2), true -> Vector(4, 4, 4))

scala> val ys = Vector(1, 1, 2, 2, 4, 4, 4).groupBy(x => x)
val ys: Map[Int,Vector[Int]] =
  Map(2 -> Vector(2, 2), 4 -> Vector(4, 4, 4), 1 -> Vector(1, 1))
```

b)

```
def freq(xs: Vector[Int]): Map[Int, Int] =
  xs.groupBy(x => x).map(p => p._1 -> p._2.size)
```

Explanation: the method `groupBy` creates a table with pairs `k, v` where `v` is a sequence with as many `k` as the number of times `k` occurs in `xs`. By converting all values `p._2` to the size `p._2.size` we get a frequency table.

```
scala> freq(toss(1000))
val res0: Map[Int,Int] =
  Map(5 -> 163, 1 -> 174, 6 -> 161, 2 -> 169, 3 -> 167, 4 -> 166)

scala> freq(toss(1000)).toVector.sortBy(_._1).foreach(println)
(1,183)
(2,167)
(3,169)
(4,179)
(5,154)
(6,148)
```

c)

```
shoppinglist.groupMap(_.department)(_.item)
```

d)

```
shoppinglist.groupMapReduce(_.department)(_.price)(_ + _)
```

**Solution Task 14.** –

# 10. Solution Excercise10

## 10.1 Basic Tasks; preparation for the lab

**Solution Task 1.** *Match concepts with descriptions.*

| | | | | |
|---|---|---|---|---|
| base type | 1 | ⇝ | N | most general type in an inheritance hierarchy |
| super type | 2 | ⇝ | O | a type that is more general |
| subtype | 3 | ⇝ | D | a type that is more specific |
| runtime type | 4 | ⇝ | J | may be more specific than static type |
| dynamic binding | 5 | ⇝ | L | runtime type determines which method executes |
| polymorphism | 6 | ⇝ | B | can have many forms, e.g., one of several subtypes |
| trait | 7 | ⇝ | I | abstract, can be mixed in, can have parameters |
| mix-in | 8 | ⇝ | H | add properties with `with` and a trait |
| overridden member | 9 | ⇝ | M | subtype member replaces supertype member |
| anonymous class | 10 | ⇝ | C | class without name, extended with extra implementation |
| protected member | 11 | ⇝ | K | visible only in subtypes |
| abstract member | 12 | ⇝ | F | has no implementation |
| abstract class | 13 | ⇝ | E | can have parameters, cannot be instantiated, cannot be mixed i |
| sealed type | 14 | ⇝ | P | subtyping outside this file is prevented |
| reference type | 15 | ⇝ | A | has supertype `AnyRef`, allocated on heap via reference |
| value type | 16 | ⇝ | G | has supertype `AnyVal`, stored directly on stack |

**Solution Task 2.** *Common base type.*

a) `Vector[Object]`. The type `Object` in the JVM corresponds to the type `AnyRef` which is the base type for all reference types.

b) Error message:

```
scala> vegetables.map(_.weight).sum
-- Error:
1 |vegetables.map(_.weight).sum
  |               ^^^^^^
  |              value weight is not a member of Object - did you mean wait?
-- Error:
1 |vegetables.map(_.weight).sum
  |                          ^
  |ambiguous implicit arguments: both object DoubleIsFractional in object Nume
```

The first error is because the elements of the vector are of type `Object` and the member `weight` is not defined for this type. The second error is a follow-up error caused by the fact that a sequence with elements of type `Object` cannot be summed because the compiler cannot infer that the element type is numeric.

c) The attribute `weight` is initialized during the construction of `Cucumber` respectively `Tomat`. The value is given by the respective class parameter.

d) `Vector[Vegetable]`.

e) Yes. Since the static type of the elements in the sequence is `Vegetable` (the dynamic type can be any subtype of `Vegetable`) and all instances of this type are

guaranteed to have the attribute `weight` of type `Int`, the compiler can at *compile time* conclude that summing is valid and thus can compile the code to executable machine code.

f)

```
scala> new Vegetable
-- Error:
1 |new Vegetable
  |    ^^^^^^^
  |    Vegetable is a trait; it cannot be instantiated
```

g)

```
scala> val anonymousVegetable = new Vegetable { val weight = 42 }
val anonymousVegetable: Vegetable = anon$1@1edde8b6
scala> anonymousVegetable.toString
val res0: String = anon$1@1edde8b6
```

The type is `Vegetable` and here becomes a so-called *anonymous class*, since we did not use a named class with **`extends`**, but only "attached" a class body within braces directly during construction. When creating anonymous classes you must use the keyword **`new`**.

The compiler invents a unique class name, here anon$1, to keep track of the anonymous class during compilation to machine code. The string representation contains a hexadecimal number that is unique for the instance, here `1edde8b6`.

h)

```
scala> new Vegetable { }
-- Error:
1 |new Vegetable { }
  |^
  |object creation impossible, since val weight: Int in trait Vegetable is not defined
```

**Solution Task 3.** *Polymorphism in inheritance, i.e. subtype polymorphism.*

a)

```
def createAnimal(): Animal =
  if math.random() > 0.5 then Cow() else Pig()
```

b)

```
class Horse extends Animal:
  def makeNoise = println("Neigh")

def createAnimal(): Animal =
   math.random() match
    case r if r < 0.33 => Cow()
    case r if r < 0.67 => Pig()
    case _             => Horse()
```

**Solution Task 4.** *Various types of integer pairs for the lab `snake0`.*

a)

```
trait Pair[T]:
  def x: T
  def y: T
  def tuple: (T, T) = (x, y)
```

b)

- Does the above code also work without keyword **override**? Why?

  Yes, it works because **override** is not required when implementing inherited *abstract* members in a subtype. Abstract members lack implementation and there is nothing to override.

- When must **override** be used?

  It is required if you want to give an inherited member a *different* implementation in the subtype, if this member already has an implementation in the supertype. Your new implementation overrides (replaces) the inherited member's implementation.

- What are the advantages and disadvantages of using **override** even when not needed?

  *Advantage:* the compiler helps check that you are actually implementing an inherited member and not e.g. misspelled the member's name.

  *Disadvantage:* more to type and thus longer to read.

c)

```
case class Dim(x: Int, y: Int) extends Pair[Int]
object Dim:
  def apply(dim: (Int, Int)): Dim = Dim(dim._1, dim._2)
```

d)

```
case class Pos private (x: Int, y: Int, dim: Dim) extends Pair[Int]:
  def +(p: Pair[Int]): Pos = Pos(x + p.x, y + p.y, dim)
  def -(p: Pair[Int]): Pos = Pos(x - p.x, y - p.y, dim)

object Pos:
  def apply(x: Int, y: Int, dim: Dim): Pos =
    import java.lang.Math.floorMod as mod
    new Pos(mod(x, dim.x), mod(y, dim.y), dim) //NOTE: new is necessary here!

  def random(dim: Dim): Pos =
    import scala.util.Random.nextInt as rni
    Pos(rni(dim.x), rni(dim.y), dim)
```

e) If you forget to write **new** explicitly in the companion object's `apply` method, it becomes a recursive call resulting in an infinite loop at runtime. With **new**, it is guaranteed that the private primary constructor for `Pos` is called.

In `Dim.apply` the parameter types differ between the factory method and the primary constructor, and the compiler chooses the primary constructor since it matches the given two separate integers and not a 2-tuple.

f)

```
enum Dir(val x: Int, val y: Int) extends Pair[Int]:
  case North extends Dir( 0, -1)
  case South extends Dir( 0,  1)
  case East  extends Dir( 1,  0)
  case West  extends Dir(-1,  0)
export Dir.*  // makes North etc visible within package snake
```

**Solution Task 5.** *Supertype with parameter.*

a)

```
val person = new Person("Person1")
val academic = new Academic("Person2", "LTH")
val student = new Student("Person3", "LTH", "D")
val researcher = new Researcher("Person4", "LTH", "PhD Student")
```

b)

```
val vec = Vector(person, academic, student, researcher)
for(i <- vec){ print(i.toString + i.name) }
```

c) Error message when instantiating `abstract class Academic`: `Academic is abstract; it cannot be instantiated` It does *not* work equally well with a `trait` in the special case of `Academic`, since a trait may not forward parameters to a supertype. Error message: `trait Academic may not call constructor of trait Person`

```
trait Person(val name: String)
abstract class Academic(
name: String,
val university: String) extends Person(name)
class Student(
name: String,
university: String,
program: String) extends Academic(name, university)
class Researcher(
name: String,
university: String,
title: String) extends Academic(name, university)
```

d)

```
scala>
|trait Person(val name: String)
|
| abstract class Academic(
| name: String,
| val university: String) extends Person(name)
|
| case class Student(
| name: String,
| university: String,
| program: String) extends Academic(name, university)
```

```
|
| case class Researcher(
| name: String,
| university: String,
| title: String) extends Academic(name, university)
-- Error:
8 | name: String,
| ^
| error overriding value name in trait Person of type String;
| value name of type String needs override modifier
-- Error:
9 | university: String,
| ^
| error overriding value university in class Academic of type String;
| value university of type String needs override modifier
-- Error:
13 | name: String,
| ^
| error overriding value name in trait Person of type String;
| value name of type String needs override modifier
-- Error:
14 | university: String,
| ^
| error overriding value university in class Academic of type String;
| value university of type String needs override modifier
```

```
trait Person(val name: String)
abstract class Academic(
  name: String,
  val university: String) extends Person(name)
case class Student(
  override val name: String,
  override val university: String,
  program: String) extends Academic(name, university)
case class Researcher(
  override val name: String,
  override val university: String,
  title: String) extends Academic(name, university)
```

```
scala> val ps = Vector(Student("Kim", "Lund", "CS"), Researcher("Herz", "Lund", "Dr"))
val ps: Vector[Academic] = Vector(Student(Kim,Lund,CS), Researcher(Herz,Lund,Dr))
scala> ps :+ new Person("Abstract") {}
val res0: Vector[Person] =
  Vector(Student(Kim,Lund,CS), Researcher(Herz,Lund,Dr), anon1@1941bbf3)
```

e)

```
trait Person:
  val name: String
  val nbr: Long
trait Academic extends Person:
  val university: String
case class Student(
  name: String,
```

```
  nbr: Long,
  university: String,
  program: String) extends Academic
case class Researcher(
  name: String,
  nbr: Long,
  university: String,
  title: String) extends Academic
case class NonAcademic(
  name: String,
  nbr: Long) extends Person
```

## 10.2 Extra Tasks; additional practice

**Solution Task 6.** *Base type* `Shape` *and subtypes* `Rectangle` *and* `Circle`.

a)

```
val c1 = Circle(Point(1, 1), 42)
val r1 = Rectangle(Point(3, 3), 20, 30)
c1.move(2, 3)
r1.move(3, 2)
```

b)

```
case class Point(x: Double, y: Double):
  def move(dx: Double, dy: Double): Point = Point(x + dx, y + dy)
  def moveTo(x: Double, y: Double): Point = Point(x, y)

trait Shape:
  def pos: Point
  def move(dx: Double, dy: Double): Shape
  def moveTo(x: Double, y: Double): Shape

case class Rectangle(pos: Point, width: Double, height: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))

case class Circle(pos: Point, radius: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))
```

c) `def distanceTo(that: Point): Double = math.hypot(that.x - x, that.y - y)`

d) `def distanceTo(that: Shape): Double = pos.distanceTo(that.pos)`.

e)

```
case class Point(x: Double, y: Double):
  def move(dx: Double, dy: Double): Point = Point(x + dx, y + dy)
  def moveTo(x: Double, y: Double): Point = Point(x, y)
  infix def distanceTo(that: Point): Double = math.hypot(that.x - x, that.y - y)

trait Shape:
  def pos: Point
  def move(dx: Double, dy: Double): Shape
```

```
  def moveTo(x: Double, y: Double): Shape
  infix def distanceTo(that: Shape): Double = pos.distanceTo(that.pos)

case class Rectangle(pos: Point, width: Double, height: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))

case class Circle(pos: Point, radius: Double) extends Shape:
  def move(dx: Double, dy: Double): Shape = copy(pos = pos.move(dx, dy))
  def moveTo(x: Double, y: Double): Shape = copy(pos.moveTo(x, y))
```

## 10.3 Advanced Tasks; challenges

**Solution Task 7.** *Mixins.*

a) There are many ways, here are some example solutions:

```
val numberOfFlyingDucks: Int =
  flock.count(f => f.isInstanceOf[Duck] && f.isAbleToFly)
```

```
val numberOfFlyingDucks: Int =
  flock.filter(f => f.isInstanceOf[Duck] && f.isAbleToFly).size
```

```
val numberOfFlyingDucks: Int =
  flock.collect{case f: Duck if f.isAbleToFly}.size
```

```
val numberOfFlyingDucks: Int = flock.map(_ match
  case f: Duck if f.isAbleToFly => 1
  case _ => 0
).sum
```

b)

```
val numberOfCrows: Int = flock.filter(f => !f.isAbleToSwim).size * 2
val numberOfDucks: Int = flock.filter(f => f.isAbleToSwim).size * 4
```

**Solution Task 8.** *Final classes.*

a) Put `final` before `class` in the classes.

b) error: illegal inheritance from final class Crow.

**Solution Task 9.** *Access rules in inheritance and the keyword* `protected`.

a)

```
2 |  def reveal = myHome
  |                ^^^^^^^^^
  |                Not found: myHome
```

b)

```
scala> class Sub extends Super:
         def cryptic = ourHome * math.Pi
scala> (new Sub).ourHome
-- Error:
1 |(new Sub).ourHome
  |^^^^^^^^^^^^^^^^^^
  |value ourHome in trait Super cannot be accessed as a member of Sub.
  | Access to protected value ourHome not permitted because enclosing object
  | is not a subclass of trait Super where target is defined
```

c) Yes.

**Solution Task 10.** *Using* ***protected****.*

a) In Bird:

```
protected var soundCount: Int = 0
def makeNoise: Unit = { print(sound * 2); soundCount += 2 }
```

In Duck: `override def makeNoise = { print(sound * 4); soundCount += 4 }`

b) `def numberOfSounds: Int = soundCount`

c) If a class representing a bird that would produce more/fewer sounds than a typical `Bird` needs to change `makeNoise`, that method needs access to `soundCount`, which cannot be **`private`**.

d) The counter variable should not be modifiable from any other part of the program.

**Solution Task 11.** *Mixin of properties.*

```
trait Bird:
  val sound: String
  def makeNoise: Unit = { print(sound * 2); soundCount += 2 }
  protected var soundCount: Int = 0
  val canSwim: Boolean
  val canFly: Boolean
  val isBig: Boolean
  def numberOfSounds: Int = soundCount

trait CanSwim { val canSwim = true }
trait CannotSwim { val canSwim = false }
trait CanFly { val canFly = true }
trait MaybeCanFly { val canFly = math.random() < 0.8 }
trait MaybeCanSwim { val canSwim = math.random() < 0.2 }
trait IsBig { val isBig = true }
trait IsSmall { val isBig = false }

final class Crow extends Bird, CanFly, CannotSwim, IsBig:
  val sound = "caw"

final class Duck extends Bird, CanSwim, MaybeCanFly, IsBig:
  val sound = "quack"
  override def makeNoise = { print(sound * 4); soundCount += 4 }
```

```
final class Chick extends Bird, CanFly, MaybeCanSwim, IsSmall:
  val sound = "chirp"
  override def makeNoise = { print(sound * 8); soundCount += 8 }
```

In REPL:

```
val birds = Vector.fill(42)(
  if math.random() < 0.33 then Crow()
  else if math.random() < 0.5 then Duck()
  else Chick()
)
fyle.filter(f => f.isInstanceOf[Crow]).size * 2
fyle.filter(f => f.isInstanceOf[Duck]).size * 4
fyle.filter(f => f.isInstanceOf[Chick]).size * 8
```

# 11. Solution `Excercise11`

## 11.1 Basic Tasks; preparation for the lab

**Solution Task 1.** *Context parameter.*

a)

```
scala> given Int = 0
val res0: Int = 83

scala> f(using 41)
val res1: Int = 42

scala> g(41)(using 42)
val res2: Int = 83
```

If you forget **`using`** when giving an explicit context argument, it results in a compilation error. The compiler becomes "confused" and thinks you are trying to provide a "regular" argument to a (in this case) nonexistent regular parameter list.

```
scala> f(41)
-- [E050] Type Error: ----------------------------------------------
1 |f(41)
  |^
  |method f does not take more parameters
  |
  | longer explanation available when compiling with `-explain`
1 error found

scala> :setting -explain

scala> f(41)
-- [E050] Type Error: ----------------------------------------------
1 |f(41)
  |^
  |method f does not take more parameters
  |----------------------------------------------------------------
    | Explanation (enabled by `-explain`)
  |- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
  | You have specified more parameter lists than defined in the
    method definition(s).
   ----------------------------------------------------------------

scala> g(41)(42)
-- [E050] Type Error: ----------------------------------------------
1 |g(41)(42)
  |^^^^^
  |method g does not take more parameters
  |----------------------------------------------------------------
    | Explanation (enabled by `-explain`)
  |- - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - -
  | You have specified more parameter lists than defined in the
    method definition(s).
   ----------------------------------------------------------------
```

It is uncommon to specify **`using`** parameters explicitly; it is more common to let the compiler summon a given value.

b) The given value `0` is bound to the corresponding context parameter, which should be declared in a separate parameter list starting with **`using`**.

```
scala> f
val res3: Int = 1

scala> g(42)
val res4: Int = 42
```

c) No, it results in a compilation error if you try to mix regular and context parameters in the same parameter list:

```
scala> def h(i: Int, using j: Int) = i + j
-- [E040] Syntax Error: ----------------------------------------------
1 |def h(i: Int, using j: Int) = i + j
  |                     ^
  |                     ':' expected, but identifier found
```

It is a deliberate design choice to require separate parameter lists so that there is no confusion about whether a regular parameter or context parameter is meant.

**Solution Task 2.** *Multiple given values in local context.*

a) 2

b) 43

c) When the compiler tries to summon a given value to automatically use as argument to the **`using`** parameter `dx`, it searches first in the context nearest to the call. If a given value exists in the companion object for the parameter type, the compiler takes this as a last resort if no other given value is found closer to the call.

**Solution Task 3.** *Solution to the configuration problem using given values.*

Below is a test of the three different solutions given in the task. a)

After each test we discuss associated pros and cons, as requested in the task. b)

```
def testGlobalVar(useDefault: Boolean = true) =
  import GlobalVar.*
  if useDefault then println(greetMsg) else
    GreetConfig.config = GreetConfig("Good morning", "world")
    println(greetMsg)
```

Since `config` here is a mutable variable, a change in one place can affect completely different parts of the program, which sometimes can be an advantage, but often a disadvantage because it can be hard to understand what is happening by reading only one part of the program – a change of `config` can occur anywhere. It is easy to forget to change back to the default value if that is what is expected.

```
scala> testGlobalVar(); testGlobalVar(false); testGlobalVar()
Hello World!
Good morning world!
Good morning world!
```

```
def testDefaultArgs(useDefault: Boolean = true) =
  import DefaultArgs.*
```

```
3   if useDefault then println(greetMsg()) else
4     println(greetMsg(GreetConfig("Godd morning","world")))
```

Here there is no state change and default usage is simple, but it is not easy to deviate from the default that applies in a local context; at *each* individual call you have to explicitly supply all the arguments that should not be default, as shown on line 4 above. Changing the default has only local impact. If all arguments are to follow the default, then you must not forget to call with empty parentheses: `greetMsg()`. (What happens otherwise?)

```
scala> testDefaultArgs(); testDefaultArgs(false); testDefaultArgs()
Hello World!
Good morning world!
Hello World!
```

With context parameters the flexibility is greater; **using** parameters let the user control what applies in different contexts and the call becomes simple regardless of whether the default value or other, locally given values are desired. Changing default has only local impact, but this affects arbitrarily many calls in the local context – arguments that differ can be given once without needing to be repeated at each call. For calls where you want the compiler to summon given values for the context parameter no parentheses should be used, making the calls short and simple.

```
def testGivenVal(using g: GivenVal.GreetConfig) = println(g.greetMsg)
```

```
scala> testGivenVal
Hello World

scala> def localContext =
         import GivenVal.*
         given GreetConfig = GreetConfig("Good morning","world")
         testGivenVal

scala> localContext
Good morning world

scala> testGivenVal
Hello World
```

c) The compiler summons a given value in the local context:

```
scala> summon[GivenVal.GreetConfig]
val res0: GivenVal.GreetConfig = GreetConfig(Hello,World)
```

The compiler follows this priority order when searching for a unique given value:

1. **Explicit** arguments to context parameters marked with **using**
2. **given** and **import given** ... in the current scope
3. **given** values in the **companion objects** for the used type.

If multiple given values can be summoned for types in a common inheritance hierarchy, the compiler chooses the given value of the *most general* type.

d) It results in a compilation error if the compiler does not find a given value for the type in question.

```
scala> summon[Long]
-- Error: ----------------------------------------------------------------
1 |summon[Long]
  |            ^
  |            no given instance of type Long was found for parameter x of
               method summon in object Predef
1 error found
```

e) Yes! It must *not* be ambiguous which given value should be summoned:

```
scala> def ambiguous =
     |   given a: Int = 42
     |   given b: Int = 43
     |   summon[Int]
     |
-- Error: ----------------------------------------------------------------
4 |  summon[Int]
  |             ^
  |ambiguous given instances: both given instance b and given instance a
  |match type Int of parameter x of method summon in object Predef
1 error found
```

Read more about contextual abstractions here:
https://docs.scala-lang.org/scala3/reference/contextual/

## 11.2 Extra Tasks; additional practice

**Solution Task 4.** *Context parameter and given value.*

a)

```
scala> add(1)
-- Error: ----------------------------------------------------------------
1 |add(1)
  |      ^
  |    no given instance of type Int was found for parameter y of method add
1 error found
```

b) Now there is a given value that the compiler can automatically fill in at the call site.

c) `def sub(x: Int)(using y: Int) = x - y`

## 11.3 Advanced Tasks; challenges

**Solution Task 5.** *Variance and type bounds.*

a) Make the box flexible in its type parameter with a + before the type parameter as below.

```
case class Box[+A](x: A)
```

The compiler applies the rules for covariance because the type parameter has a plus sign: `Box[Cat]` is a subtype of `Box[Any]` if `Cat` is a subtype of `Any`, which it is since all types are subtypes of `Any`.

b) The explanation for behavior concerns different variance:

- The collection `Vector` is covariant and thus flexible in its type parameter (as are other immutable sequences in Scala's standard library). The compiler thus considers `Vector[Cat]` as a subtype of `Vector[Pet]` since `Cat` is a subtype of `Pet`. Where a `Vector[Pet]` is expected, `Vector[Cat]` conforms and is accepted.

- A set has an apply method from element type to `Boolean` which checks membership. Hence `Set[T]` inherits `Function1[T, Boolean]` which is declared contravariant in `T`, allowing a set to be used where a function `T => Boolean` is expected. Even though it would be practical if `Set[T]` were covariant like `Vector`, `List`, `Seq` etc., a type parameter cannot be both covariant and contravariant at the same time. Therefore `Set` is invariant and sets are not flexible in their type parameter. Thus `Set[Cat]` is *not* a subtype of `Set[Pet]` even though `Cat` is a subtype of `Pet`, which causes the compilation error in the example. See also https://stackoverflow.com/questions/676615/why-is-scalas-immutable-set-not-covariant-in-its-type

- Using `:settings -explain` the compiler outputs a longer explanation clarifying the proof performed during compilation's type checking.

c) It is a compilation error because the method `isHealthy` does not exist for arbitrary types.

d) Add an upper bound to ensure the method `isHealthy` exists for all types bound to the type parameter `A`:

```
class Vet[-A <: Pet]:
  def treat(x: A): Unit = x.isHealthy = true
```

The compiler guarantees that the type parameter `A` is "less than or equal to" `Pet`.

e) The veterinarian `Vet` is flexible in its type parameter and the minus sign indicates contravariance, meaning `Vet[Pet]` is a subtype of `Vet[Cat]` since `Cat` is a subtype of `Pet`. This can be demonstrated with the example below:

```
scala> val pinkPanther = Cat()
val pinkPanther: Cat = Cat@33e7ece5

scala> val somePet: Pet = Cat()
val somePet: Pet = Cat@57f1cb96

scala> val catVet = Vet[Cat]()
val catVet: Vet[Cat] = Vet@1060e784

scala> pinkPanther.isHealthy = false

scala> catVet.treat(pinkPanther)

scala> pinkPanther.isHealthy
val res2: Boolean = true

scala> somePet.isHealthy = false

scala> catVet.treat(somePet)
-- [E007] Type Mismatch Error: --------------
1 |catVet.treat(somePet)
  |             ^^^^^^^
```

```
  |             Found:    (somePet : Pet)
  |             Required: Cat

scala> val powerVet = Vet[Pet]()
val powerVet: Vet[Pet] = Vet@2eb90ae9

scala> pinkPanther.isHealthy = false

scala> powerVet.treat(pinkPanther)

scala> pinkPanther.isHealthy
val res3: Boolean = true

scala> val pluto = Dog()
val pluto: Dog = Dog@6f27db5d

scala> pluto.isHealthy = false

scala> powerVet.treat(pluto)

scala> pluto.isHealthy
val res4: Boolean = true
```

**Solution Task 6.** *Type classes and context parameters.*

a)

- First we declare a **`trait`**, `CanCompare`, with a generic type parameter `T`. It contains an abstract method `compare` that takes two parameters of type `T` and returns an `Int`.
- Then a method `sort` is defined, also with a generic type parameter `T`. This method takes two parameters, `a` and `b` of type `T`, and a **`using`** parameter `cc` which must be an instance of `CanCompare[T]`. Inside the method, the `compare` method from `CanCompare` is used to determine if `a` and `b` should be swapped.
- When we try to run `sort(42, 41)` we get a compiler error because there is no given instance of `CanCompare[Int]`.
- We fix this by defining **`given`** `intComparator` of type `CanCompare[Int]` overriding the `compare` method. Now the following call `sort(42, 41)` works and returns `(Int, Int) = (41, 42)`.
- When we try `sort` with `Double` arguments, we get a similar error as before because no `CanCompare` for `Double` exists.

b)

```
scala> given doubleComparator: CanCompare[Double] with
         override def compare(a: Double, b: Double): Int = (a - b).toInt
```

c)

```
scala> given stringComparator: CanCompare[String] with
         override def compare(a: String, b: String): Int = a.compareTo(b)
```

**Solution Task 7.** *Usage of given ordering.*

a) Example tests:

```
scala> isSorted(Vector(1,2,3))
val res0: Boolean = true

scala> isSorted(Vector(1,2,4,3))
val res1: Boolean = false

scala> isSorted(List(1,2,3))
val res2: Boolean = true

scala> isSorted(List(1,2,4,3))
val res3: Boolean = false
```

b)

```
scala> def isSorted[T](xs: Seq[T]): Boolean = xs == xs.sorted
-- [E172] Type Error: -----------------------------------------------------------
1 |def isSorted[T](xs: Seq[T]): Boolean = xs == xs.sorted
  |                                                     ^
  | No implicit Ordering defined for T..
  | I found:
  |
  |     scala.math.Ordering.ordered[T](/* missing */summon[scala.math.Ordering.
  |
  | But no implicit values were found that match type scala.math.Ordering.AsCon
  |
  | The following import might make progress towards fixing the problem:
  |
  |     import scala.math.Ordered.orderingToOrdered
  |
1 error found
```

The error means the compiler cannot derive an ordering for the generic type `T`.

c)

```
scala> isSorted(persons)
-- [E172] Type Error: -----------------------------------------------------------
1 |isSorted(persons)
  |                 ^
  | No implicit Ordering defined for Person..
  | I found:
  |
  |     scala.math.Ordering.ordered[Person](/* missing */summon[scala.math.Orde
  |
  | But no implicit values were found that match type scala.math.Ordering.AsCon
  |
  | The following import might make progress towards fixing the problem:
  |
  |     import scala.math.Ordered.orderingToOrdered
  |
1 error found
```

The error occurs because the compiler does not know any ordering for the `Person` class.

d) -

e) -

**Solution Task 8.** *Create your own implicit ordering with* `Ordering`.

a)

```
scala> xs.sorted
-- [E172] Type Error: -----------------------------------------------------
1 |xs.sorted
  |         ^
  | No implicit Ordering defined for Team..
  | I found:
  |
  |     scala.math.Ordering.comparatorToOrdering[Team](/* missing */summon[java
  |
  | But no implicit values were found that match type java.util.Comparator[Tean
  |
  | One of the following imports might make progress towards fixing the problem
  |
  |     import scala.math.Ordering.comparatorToOrdering
  |     import scala.math.Ordering.ordered
  |
1 error found
```

The error appears because the compiler doesn't know any ordering for `Team`.

After assigning `given Ordering[Team] = Ordering.by(t => t.rank)` there exists a given ordering for `Team` in the current context. Therefore the following call `xs.sorted` works as intended.

b) -

c) -

d) -

e)

```
scala> Team("fnatic",1499) < Team("gurka", 2)
-- [E008] Not Found Error: ------------------------------------------------
1 |Team("fnatic",1499) < Team("gurka", 2)
  |^^^^^^^^^^^^^^^^^^^^
  |value < is not a member of Team, but could be made available as an extension
  |
  |One of the following imports might fix the problem:
  |
  |  import scala.math.Ordered.orderingToOrdered
  |  import scala.math.Ordering.Implicits.infixOrderingOps
  |
1 error found
```

As the compiler says, < is not a method defined for `Team`. However, after `import Team.highestRankFir` the implicit conversions defined for `highestRankFirst: Ordering[Team]` (because it is an object of type `Ordering`) are imported. Thus, < works. See task 7e for more background.

**Solution Task 9.** *Custom ordering by inheriting from* `Ordered`

a)

```
case class Team(name: String, rank: Int) extends Ordered[Team]:
```

```
  override def compare(that: Team): Int = -rank.compare(that.rank)
```

b)

```
scala> Team("fnatic",1499) < Team("gurka", 2)
val res1: Boolean = true
```

c) Subtype polymorphism, i.e. using inheritance and interfaces builds on each subtype implementing certain members. The compiler can then guarantee that *something* will happen at calls with a subtype. However, it cannot guarantee *what* will happen, which is decided at runtime by dynamic binding.

Ad hoc polymorphism on the other hand means that the called function behaves differently for different types. This uses some form of type classes. It is the type given at the call that determines what happens, not which behavior is defined in the subtype the type "happens" to have. By compile time the compiler can thus determine what will happen based on the type, i.e. static binding. This makes it easier to reason about how the program behaves.

# 12. Solution Excercise12

## 12.1 Tasks on searching and sorting

**Solution Task 1.** *Timing measurement.*

a) Execute the code and you should find that it takes longer to find the value 1 in our Set s than in the vector v.

b)

A vector has a sequential order that `find` can use, whereas a `Set` is internally organized differently for very fast membership checks. The reason that it takes time for `find` on `Set` is that an iterator must first be created before the set can be traversed from start to end. The method `contains` on `Set` is lightning fast due to the internal structure of objects of type `Set` (which is smartly designed with so-called hash codes, allowing constant time lookup regardless of element position).

**Solution Task 2.** *Searching with built-in search methods.*

a) Suggested test of `indexOfSlice`:

```
scala> List(1,2,3,35,1,23).indexOfSlice(List(35,1,23))
res73: Int = 3
scala> List(1,2,3,35,1,23).indexOfSlice(List(35,1,3))
res74: Int = -1
```

b) Suggested test of `lastIndexOfSlice`:

```
Vector(1,2,3,4,1,2).lastIndexOfSlice(Vector(1,2))
res2: Int = 4
Vector("apa", "banan", "majs", "banan").lastIndexOfSlice(Vector("banan"))
res3: Int = 3
Vector("apa", "banan", "majs", "banan").lastIndexOfSlice(Vector("banand"))
res4: Int = -1
```

c) Note that the method `search` assumes the collection is sorted in ascending order. When we reverse the order, `search` mostly cannot find what we look for, since it will search the wrong half of the collection.

```
scala> val odd = (1 to 1000000 by 2).toVector
scala> import scala.collection.Searching._
scala> odd.search(odd.last)
res18: collection.Searching.SearchResult = Found(499999)
//Search finds the last element at position 499999 in the collection.

scala> odd.search(odd.last + 1)
res19: collection.Searching.SearchResult = InsertionPoint(500000)
//Search cannot find odd.last + 1 because it does not exist in the collection
//and thus returns an object of type InsertionPoint with value 500000.
//Our element odd.last + 1 would have been at position 500000 if it existed.

scala> odd.reverse.search(odd(0))
res20: collection.Searching.SearchResult = InsertionPoint(0)
//As explained before, search assumes the list is sorted ascending,
//so it cannot find element odd(0) = 1 when the list is reversed.

scala> def lin(x: Int, xs: Seq[Int]) = xs.indexOf(x)
```

```
scala> def bin(x: Int, xs: Seq[Int]) = xs.search(x) match
  case Found(i) => i
  case InsertionPoint(i) => -i

//Defines a method bin using the search method on a sequence.
//With pattern matching it returns only the position i,
//not an object of Found or InsertionPoint.

scala> timed{ lin(odd.last, odd) }
time: 42.294821 ms
res22: (Int, Long) = (499999,42294821)
//Finding odd.last = 499999 with linear search took ~42ms.

scala> timed{ bin(odd.last, odd) }
time: 0.147314 ms
res23: (Int, Long) = (499999,147314)
//Binary search to find 499999 took much less time.
//Because each step halves the search space.
//This is an extreme case since we search for the last element.
//Linear search for the first element would have been as fast as binary search.
```

d) It requires $log_2(n)$ comparisons. This is because at each step, the number of elements left to check is halved. After the first comparison, we have $\frac{n}{2}$ elements remaining. After the second, $\frac{n}{2^2}$, etc. When only one element remains, we have made $b$ comparisons, where:

$$\frac{n}{2^b} = 1$$

Using logarithm laws:

$$\log_2(n) = b$$

**Solution Task 3.** *Search among LTH's courses with linear search.*

a) The first line contains the column names `Kurskod KursSve KursEng Hskpoang Niva`. Then each row contains a course with data according to the column names.

b) The code downloads data and creates a vector with instances of case class `Course` using the method `fromLine`. Since variable `lth` is declared **`lazy`**, `download()` is not called until `lth` is first referenced. The number of courses is:

```
scala> val n = courses.lth.length
n: Int = 1104
```

c)

```
scala> def isCS(s: String) = s.startsWith("EDA") || s.startsWith("ETS")
scala> val x = courses.lth.find(c => isCS(c.code) && c.level == "G2")
x: Option[courses.Course] = Some(Course(EDAF05,Algoritmer, datastrukturer och
   komplexitet,Algorithms, Data Structures and Complexity,5.0,G2))
```

d)

```
def linearSearch[T](xs: Seq[T])(p: T => Boolean): Int =
   var i = 0
   while(i < xs.length && !p(xs(i))) i += 1
   if (i < xs.length) i else -1
```

e)

```
def rndCode: String =
   //randomizes from 0 to n (inclusive)
   def rnd(n: Int) = (math.random() * (n + 1)).toInt

   def letter = (rnd('Z' - 'A') + 'A').toChar
   def dig = ('0' + rnd(9)).toChar
   Seq(letter, letter, letter, letter, dig, dig).mkString
```

f)

```
val xs = Vector.fill(500000)(rndCode)
val(ixs, elapsedLin) =
  timed { xs.map(x => linearSearch(courses.lth)(_.code == x)) }
val found = ixs.filterNot(_== -1).size
```

g)

```
def linearSearch[T](xs: Seq[T])(p: T => Boolean): Int = xs.indexWhere(p)
```

**Solution Task 4.** *Search among LTH's courses with binary search.*

a) —

b)

```
def binarySearch(xs: Seq[String], key: String): Int =
  var (low, high) = (0, xs.size - 1)
  var found = false
  var mid = -1

  while (low <= high && !found) do
    mid = (low + high) / 2
    if xs(mid) == key then found = true
    else if xs(mid) < key then low = mid + 1
    else high = mid - 1
  end while
  if found then  mid else -(low + 1)
```

c) On an i7-3770K @ 3.50GHz the searches took:

- Binary search: `time: 142.6 ms`
- Linear search: `time: 3316.5 ms`

On an i7-8700T @ 2.40GHz they took:

- Binary search: `time: 81.5 ms`
- Linear search: `time: 5138.6 ms`

d) Binary search was about 23 times faster on the i7-3770K @ 3.50GHz and about 63 times faster on the i7-8700T CPU @ 2.40GHz.

**Solution Task 5.** *Insertion sort.*

a) —

b)

```
def insertionSort(xs: Seq[Int]): Seq[Int] =
  val result = scala.collection.mutable.ArrayBuffer.empty[Int]
  for e <- xs do
    var pos = 0
    while pos < result.size && result(pos) < e do pos += 1
    result.insert(pos,e)
  end for
  result.toVector
```

**Solution Task 6.** *In-place sorting.*

```
def selectionSortInPlace(xs: Array[String]): Unit =
  def indexOfMin(startFrom: Int): Int =
    var minPos = startFrom
    var i = startFrom + 1
    while (i < xs.size) do
      if (xs(i) < xs(minPos)) minPos = i
      i += 1
    end while
    minPos
  end indexOfMin

  def swapIndex(i1: Int, i2: Int): Unit =
    val temp = xs(i1)
    xs(i1) = xs(i2)
    xs(i2) = temp
  end swapIndex

  for i <- 0 to xs.size - 1 do swapIndex(i, indexOfMin(i))
end selectionSortInPlace
```

**Solution Task 7.** *Check if a sequence is sorted.*

a) The worst case takes $O(n \log n)$ comparisons for Timsort on $n$ elements. Then it requires $n$ comparisons between the sorted and unsorted list. The total comparisons in worst case is at most $n + n \log n$. For $10^6$ elements this is about $10^7$ comparisons.

```
scala> val n = 1E6
val n: Double = 1000000.0

scala> def worstCase(n: Double) = n + n * math.log(n)
def worstCase(n: Double): Double

scala> println(s"in worst case with n=$n there are ${worstCase(n)} comparisons"
in worst case with n=1000000.0 there are 1.4815510557964273E7 comparisons
```

b) A more efficient version of `isSorted` that stops searching when unsorted elements are detected:

```
def isSorted(xs: Vector[Int]): Boolean =
  if xs.length > 1 then
    var i = 0
    var result = true
    while i < xs.length-1 && result do
      if xs(i) > xs(i+1) then result = false
      i += 1
    end while
    result
  else true
end isSorted
```

c) In worst case one does $n-1$ pairwise comparisons if all elements are sorted except for the last.

d) The 2-tuple is of type `(Int, Int)`.

```
def isSorted(xs: Vector[Int]): Boolean =
  xs.zip(xs.tail).forall(x => x._1 <= x._2)
```

**Solution Task 8.** *In-place insertion sort.*

```
def insertionSort(xs: Array[Int]): Unit =
  for elem <- 1 until xs.length if xs.length > 0 do
    var pos = elem
    while pos > 0 && xs(pos) < xs(pos - 1) do
      val temp = xs(pos -1)
      xs(pos -1) = xs(pos)
      xs(pos) = temp
      pos -= 1
    end while
  end for
end insertionSort
```

**Solution Task 9.** *Sorting to new sequence with selection sort.*

```
def selectionSort(xs: Seq[String]): Seq[String] =
  def indexOfMin(xs: Seq[String]): Int = xs.indexOf(xs.min)
  val unsorted = xs.toBuffer
  val result = scala.collection.mutable.ArrayBuffer.empty[String]
  while !unsorted.isEmpty do
    val minPos = indexOfMin(unsorted)
    val elem = unsorted.remove(minPos)
    result.append(elem)
  end while
  result.toVector
end selectionSort
```

## 12.2 Tasks on threads and concurrent execution

**Solution Task 10.** *Threads.*

a) -

b) `java.lang.IllegalThreadStateException`. You cannot start a thread more than once. Therefore, you can't restart a thread when it has already executed.

c) When `start` is called, the code in `run` executes in parallel. Therefore, `Cucumber` and `Tomato` are printed interleaved. If `run` is called directly, there is no concurrent execution; `Cucumber` is printed 100 times, then `Tomato` 100 times.

d) `Thread.sleep` does not pause the thread exactly as specified. So it will usually print `zzz snark hi!`, but this is not guaranteed.

**Solution Task 11.** *Concurrent variable update.*

a) In `spendSave` the balance is fetched, changed, and stored — delayed — repeated. If `teddybear` finishes loading, changing, and storing before `ing` does the same, there is a problem, because they compete in a (eng. *race condition*). The problem is that before a thread stores the changed value, the other thread loads the old value. Only one thread 'wins' the race to store its updated value, causing the other's change to be lost. `ing` and `teddybear` thus become upset because not all their withdrawals and deposits are recorded.

**Solution Task 12.** *Thread-safe `AtomicInteger`.*

Now the `grandma` thread is guaranteed to be able to load the balance, withdraw/change, and store before the `wolf` thread can overwrite the result. In `spendSave` the thread pauses for a millisecond so the `wolf` thread can withdraw before `grandma` deposits again and the balance becomes negative. However, all withdrawals and deposits are recorded because the operations are atomic and the balance will be restored to zero without lost deposits.

**Solution Task 13.** *Concurrent execution with `scala.concurrent.Future`.*

a) error: Cannot find an implicit ExecutionContext. Future needs an ExecutionContext to run. `f` has type Future[Unit].

b) The function `printLater` returns a Future, meaning when both `printLater` and `println` are called in the loop, they run concurrently. Since starting a Future takes

longer, the `println` prints "all launched!" first, then numbers 1 to 42 print with irregular intervals as the threads sleep varying amounts.

c) `big` is a Future[Int]. The large number has 7,520,383 digits. `r` is of type Try[Int] (see Future documentation if unsure).

d) Since execution blocks until a result is received, you cannot continue typing in REPL while computation is in progress. If you wait too short, you get a TimeOutException and the computation is aborted.

**Solution Task 14.** *Use `Future` to do multiple things concurrently.*

a) -

b) -

c) Each page is delayed between 2 and 3 seconds (2000-3000 ms). So on average, each page takes 2.5 seconds to load. The vector must be filled before execution proceeds, so all 10 pages load before the first page is visible. So it takes about 2.5 x 10 = 25 seconds.

d) `f` returns a Vector filled with strings where each element is a line of the webpage. Since `f` runs in background, the program can continue while the contents compute. You can print `f` but the computation may not be done yet.

e) Same as above, but each element is itself a vector of strings.

f) Loads data in parallel, so downloads happen simultaneously but at varying speeds because of method `seg`.

g) Since data loads in parallel threads without blocking, they finish in the order threads complete, not sequentially. Eventually all are done and the result vector shows `true` values.

h) Method `happy` is well-defined and throws no exceptions. It prints `:)`. Method `unhappy` throws `java.lang.ArithmeticException` due to division by zero. This exception is caught in the callback and `:(` plus the exception is printed.

## 12.3 Extra Tasks; additional practice

**Solution Task 15.**

```
def isPrime(n: BigInt): Boolean = n match
  case _ if (n <= 1) => false
  case _ if (n <= 3) => true
  case _ if n % 2 == 0 || n % 3 == 0 => false
  case _ =>
    var i = BigInt(5)
    while i * i < n do
      if (n % i == 0 || n % (i + 2) == 0) false
      i += 6
    end while
    true
end isPrime

import scala.concurrent.*
import ExecutionContext.Implicits.global

val primes = Vector.fill(10)(Future{nextPrime(randomBigInt(16))})
```

```
primes.foreach(_.onSuccess{case i => println(i)})
```

**Solution Task 16.** *Answer theory questions.*

a) Stackoverflow gives the following explanation:
A thread is an independent set of values for the processor registers (for a single core). Since this includes the Instruction Pointer (aka Program Counter), it controls what executes in what order. It also includes the Stack Pointer, which had better point to a unique area of memory for each thread or else they will interfere with each other.

b)

```
val thread = new Thread(new Runnable{
  def run(){println(''This is a thread'')}
})
```

c) `thread.start`

d) Race conditions can occur in which one thread's resource updates overwrite another's, causing data loss.

e) Thread-safe code means multiple threads can run concurrently without incorrect results. For example, mutable variables modified by multiple threads must be handled carefully.

f) For example, you avoid explicitly creating `Thread` instances by placing code inside a Future, which also handles much under the hood for the programmer.

**Solution Task 17.** –

**Solution Task 18.** *Create your own multithreaded web server.*

a) `abbasillen` is printed backwards as `nellisabba`.

b)

c)

d)

e)

f)

g)

h)
Solution proposal:

```
package fibserver.threaded.memcache.whileloop

import java.net.{ServerSocket, Socket}
import java.io.OutputStream
import java.util.Scanner
import scala.util.{Try, Success, Failure}
import scala.concurrent._
import ExecutionContext.Implicits.global

object html:
  def page(body: String): String =  //minimal web page
    s"""<!DOCTYPE html>
       |<html><head><meta charset="UTF-8"><title>Min Sörver</title></head>
       |<body>
```

```
      |$body
      |</body>
      |</html>
      """.stripMargin

  def header(length: Int): String = //standardized header of reply to client
    s"HTTP/1.0 200 OK\nContent-length: $length\nContent-type: text/html\n\n"

  def insertBreak(s: String, n: Int = 80): String =
    if s.size < n then s else s.take(n) + "</br>" + insertBreak(s.drop(n),n)

object compute:
  import java.util.concurrent.ConcurrentHashMap
  val memcache = new ConcurrentHashMap[BigInt, BigInt]

  def fib(n: BigInt): BigInt =
    if memcache.containsKey(n) then
      println("CACHE HIT!!! no need to compute: " + n)
      memcache.get(n)
    else
      println("cache miss :( must compute fib:  " + n)
      val f = superFib(n)
      memcache.put(n, f)
      f

  private def superFib(n: BigInt): BigInt =
    if n <= 0 then 0
    else if n == 1 || n == 2 then 1
    else
      var secondLast: BigInt = 1
      var last: BigInt = 1
      var sum: BigInt = secondLast + last
      var i = 3
      while i < n do
        if memcache.containsKey(i) then
          sum = memcache.get(i)
        else
          secondLast = last
          last = sum
          sum = secondLast + last
          memcache.put(i, sum)
        i += 1
      sum


object start:

  def fibResponse(num: String) =
    num.toIntOption match
      case Some(n) => html.page(s"fib($n) == " + compute.fib(n))
      case None    => html.page(s"FEL: skriv ett heltal, inte $num")

  def errorResponse(uri:String) = html.page(s"Error: $uri </br> use /fib/heltal")

  def handleRequest(cmd: String, uri: String, socket: Socket): Unit =
    val os = socket.getOutputStream
    val afterSlash = uri.toString.drop(1) // skip initial slash
```

```
    println(s"afterSlash:$afterSlash")
    val response: String =
      if afterSlash.startsWith("fib/") then fibResponse(afterSlash.stripPrefix("fib/"))
      else errorResponse(uri)
    os.write(html.header(response.size).getBytes("UTF-8"))
    os.write(response.getBytes("UTF-8"))
    os.close
    socket.close
  end handleRequest

  def serverLoop(server: ServerSocket): Unit =
    println(s"http://localhost:${server.getLocalPort}/hej")
    while true do
      Try {
        var socket = server.accept  // blocks thread until connect
        val scan = new Scanner(socket.getInputStream, "UTF-8")
        val (cmd, uri) = (scan.next, scan.next)
        println(s"Request: $cmd $uri")
        Future { handleRequest(cmd, uri, socket) }.recover {
          case e => println(s"Reqest failed: $e")
        }
      }.recover{ case e: Throwable => s"Connection failed: $e" }

  def main(args: Array[String]) =
    val port = Try{ args(0).toInt }.getOrElse(8089)
    serverLoop(new ServerSocket(port))
```

# 13. Solution Excercise13

**Solution Task 1.** *Complete your project.* —

**Solution Task 2.** *Take a final exam.* —

**Solution Task 3.** *Prepare your project presentation.* –

**Solution Task 4.** *Create documentation for your project.*–

**Solution Task 5.** *Review exercises and laboratory work.* –